四川大學

数字逻辑实验

NEW

Part one

硬件介绍

FPGA

FPGA是一种可以重构电路的芯片，是一种硬件可重构的体系结构。

英文全称：Field Programmable Gate Array，中文名：现场可编程门阵列。

现场可编程门阵列：通过硬件语言写数字逻辑电路，FPGA在用程序实现数字电路的功能！

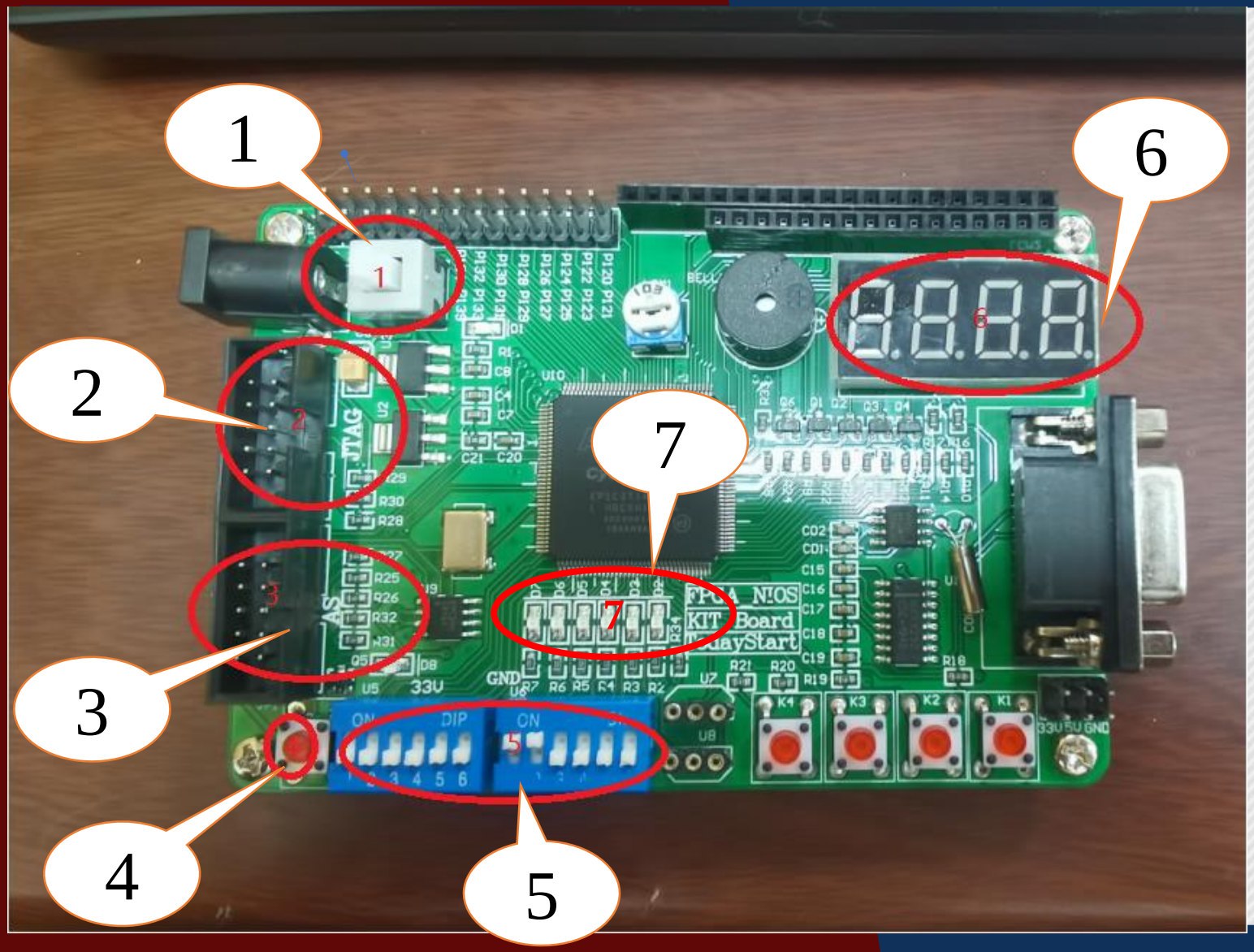
优点：

1. FPGA由逻辑单元、RAM、乘法器等硬件资源组成。通过将这些硬件资源合理组织，可实现乘法器、寄存器、地址发生器等硬件电路。
2. FPGA可无限地重新编程

缺点：

1. FPGA的所有功能均依靠硬件实现，无法实现分支条件跳转等操作
2. FPGA只能实现定点运算

FPGA开发板示意图



1. 电源开关
2. JTAG接口：电可擦，**断电之后程序自动删除。**数据写在FPGA主芯片中
3. AS接口：电可擦，**断电后程序仍然保存。**数据写在EPCS4芯片当中。
4. 重置键：还原原始程序，为FPGA芯片重新分配数据
5. 电位控制键：控制电流
6. 显示器：可以发光显示数字。下面的原点为小数点，默认会亮起
7. LED灯：可以作为输出

NEW

Part two

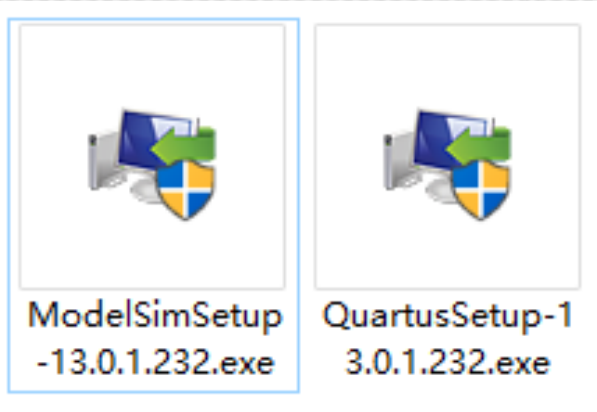
软件使用

软件使用

Quartus II 安装

Quartus II + ModelSim-Altera

注意：安装路径不能有中文！！！



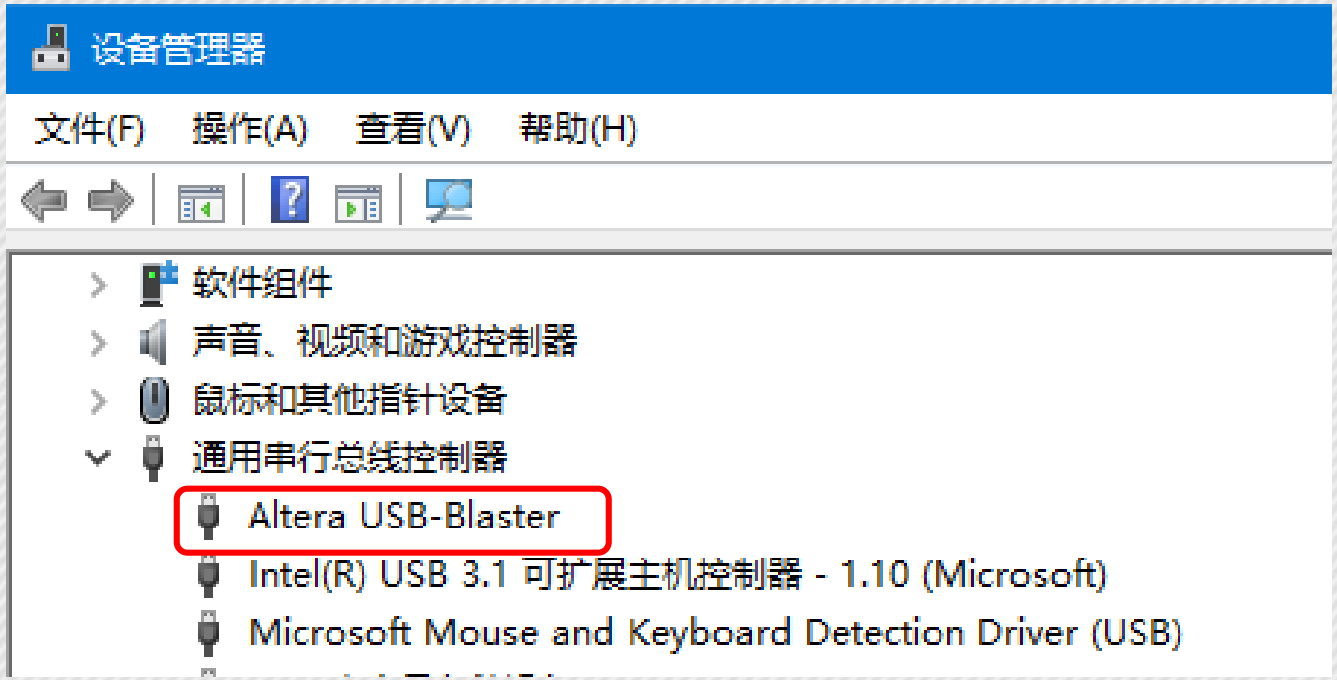
软件下载地址（版本13.0sp1 ）：
https://www.intel.com/content/www/us/en/software-kit/711920/intel-quartus-ii-subscription-edition-design-software-version-13-0sp1-for-windows.html

软件使用

USB-Blaster驱动安装

USB-Blaster在设备插上后自动安装，装好后能在设备管理器里面看到设备正常，如下图。如果有异常请查看**《USB+Blaster驱动安装不上的问题的解决.pdf》**

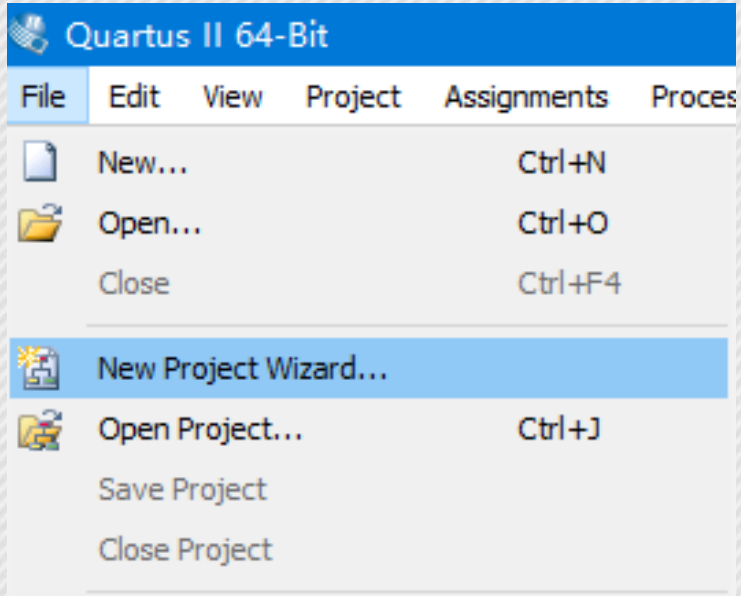
*** 在Altera的安装目录下自带驱动软件**



创建新工程

创建新工程

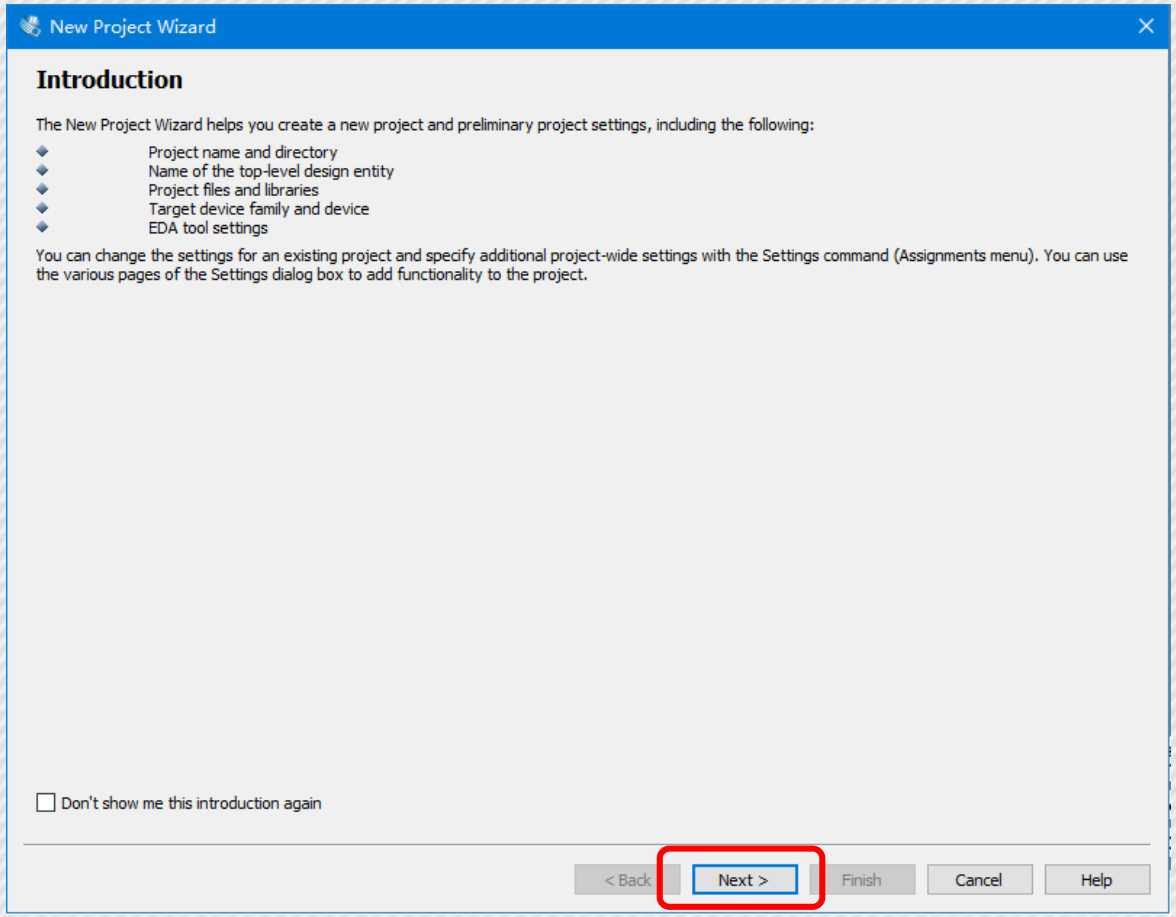
1.打开 Quartus II 设计开发软件,选择 **New Project Wizard**



软件使用

创建新工程

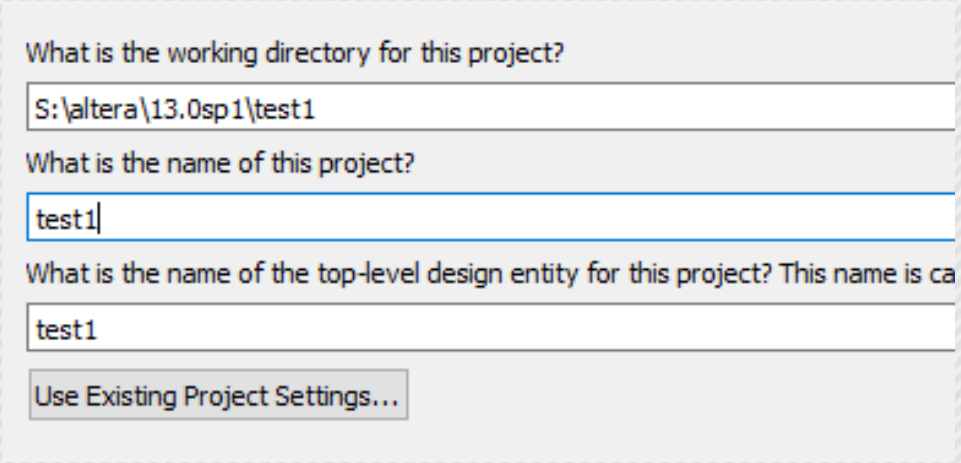
2.在弹出的创建新工程的界面中,点击 **Next**，开始创建新工程。

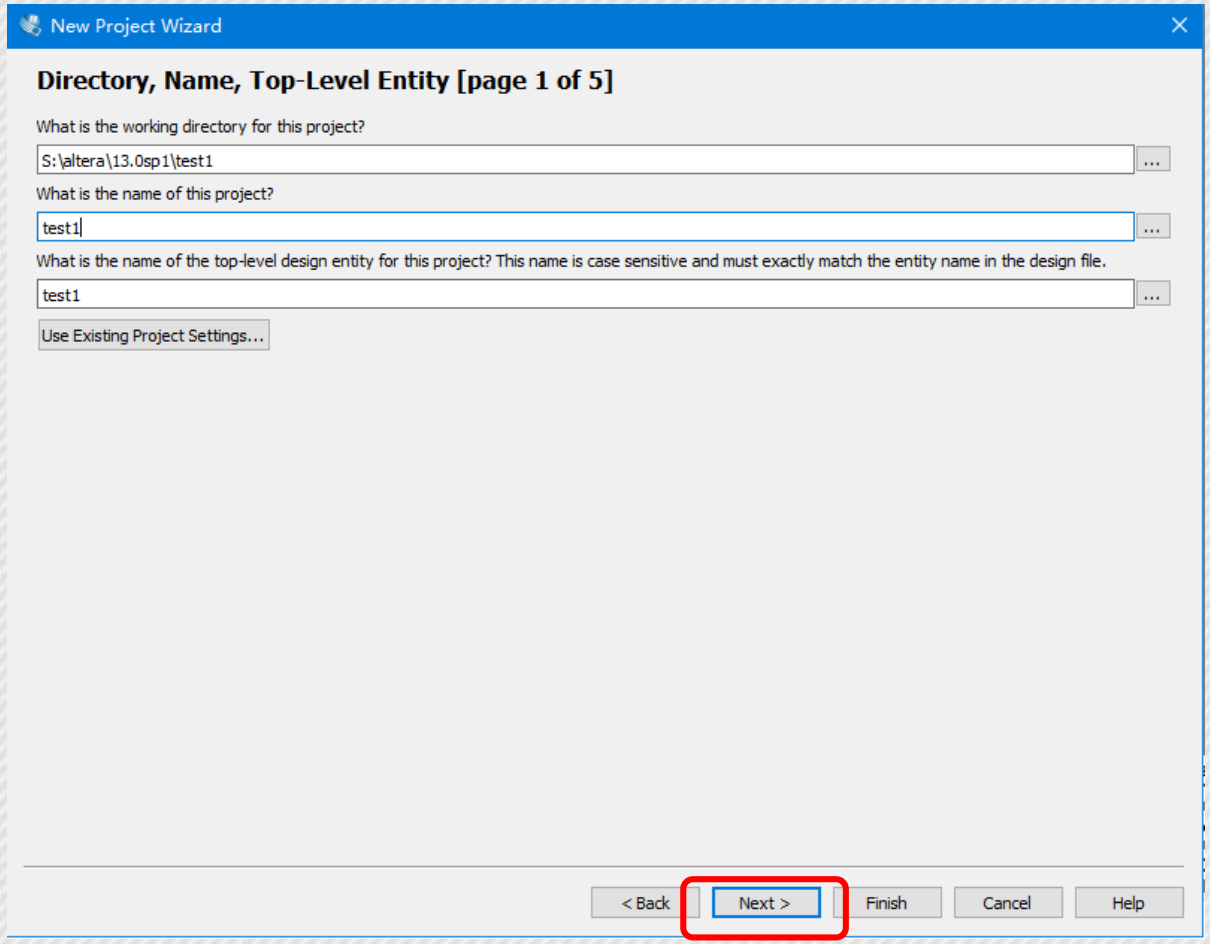


软件使用

创建新工程

3.填写工程名称，设置工程存放路径，点击 Next。（注意：路径以及工程名必须是英文。）

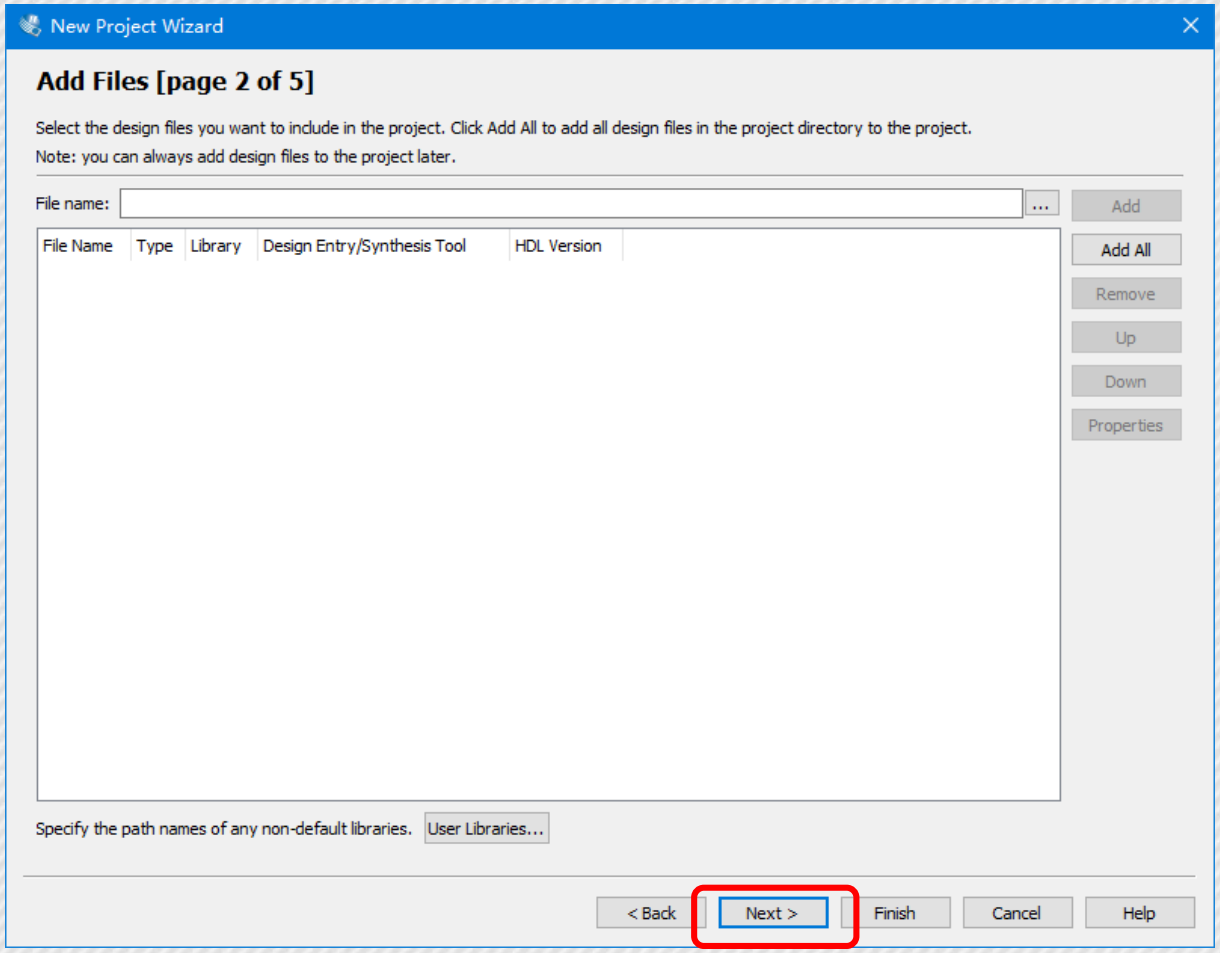




软件使用

创建新工程

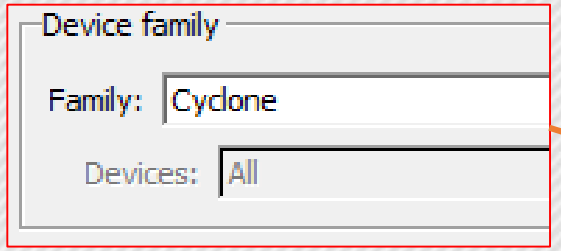
4.无需添加文件，点击 Next。

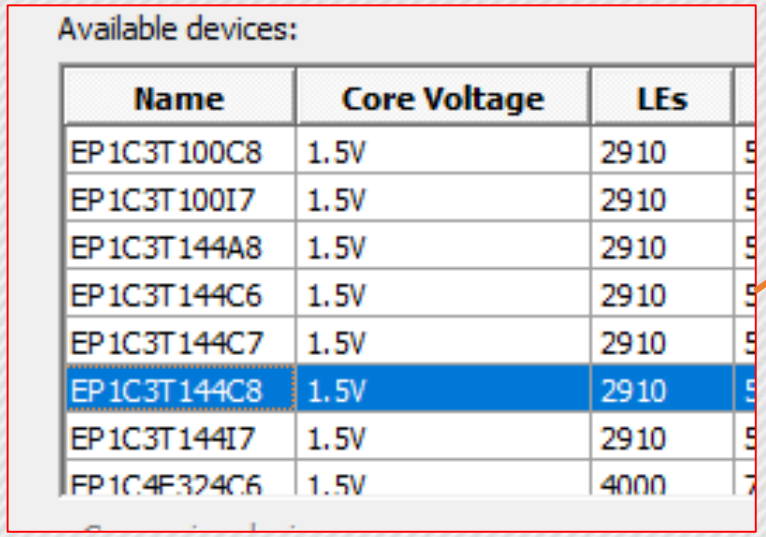


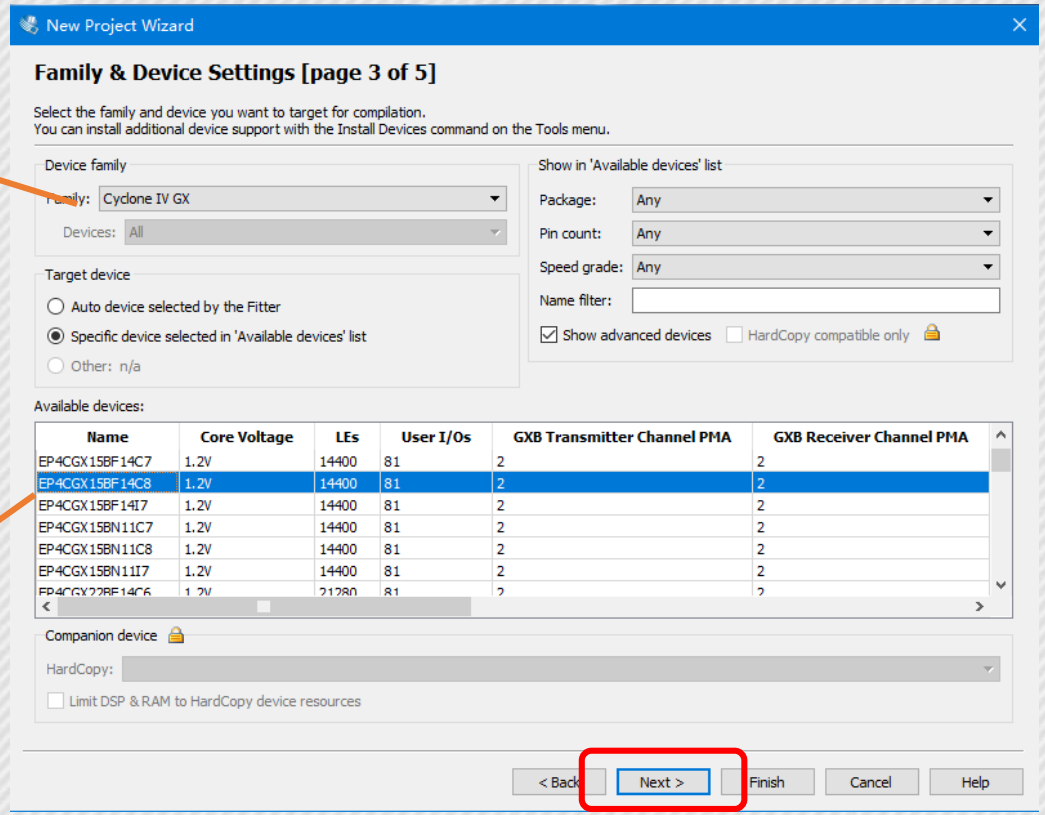
软件使用

创建新工程

5.在器件板卡选型界面中， **选择对应板卡的器件**，点击 **Next**。



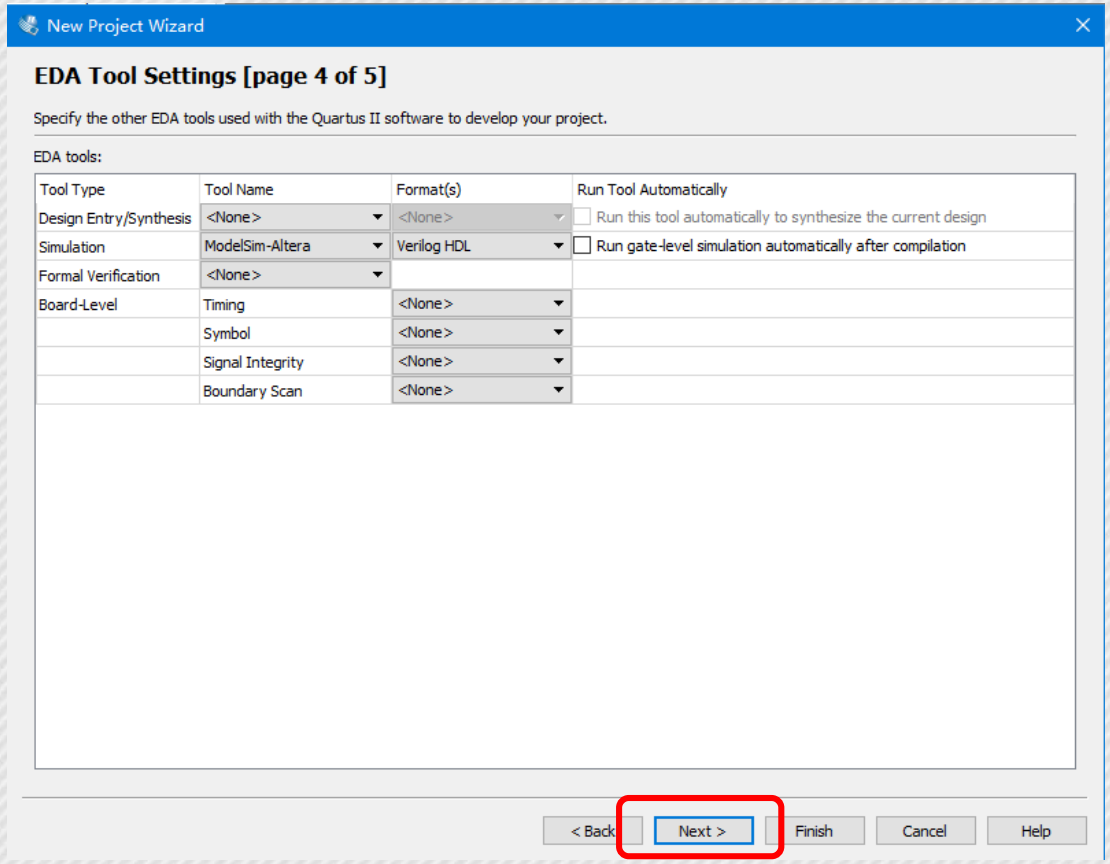


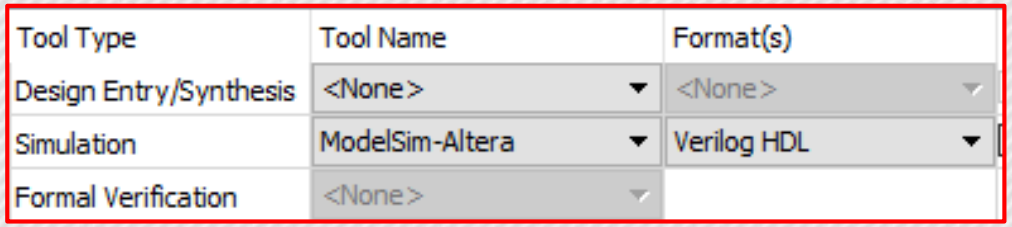


软件使用

创建新工程

6.在仿真配置界面中， 选择ModelSim-Altera 和 Verilog HDL，点击 Next。

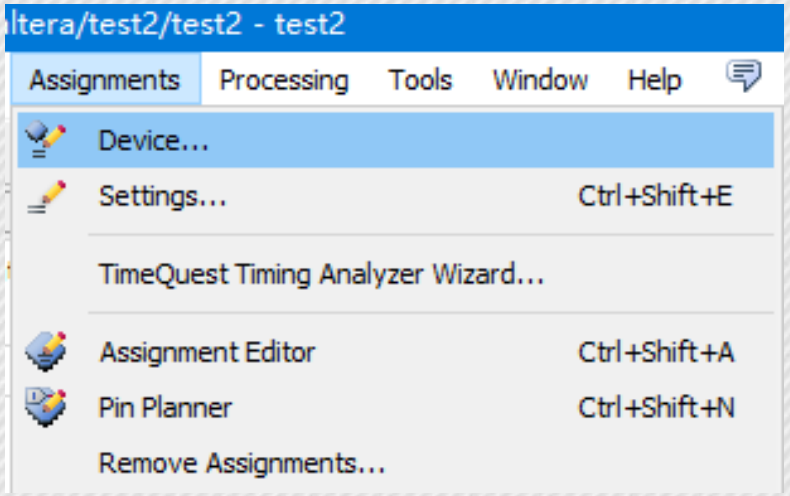


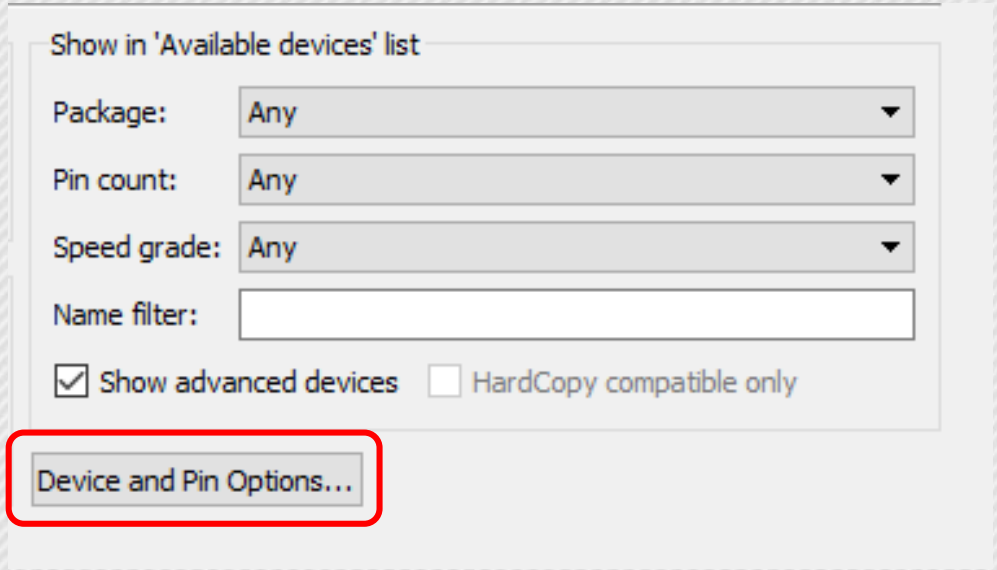


软件使用

设置未用管脚

6. 点击Assignments-Device，在弹出窗口中点击Device and Pin Options。

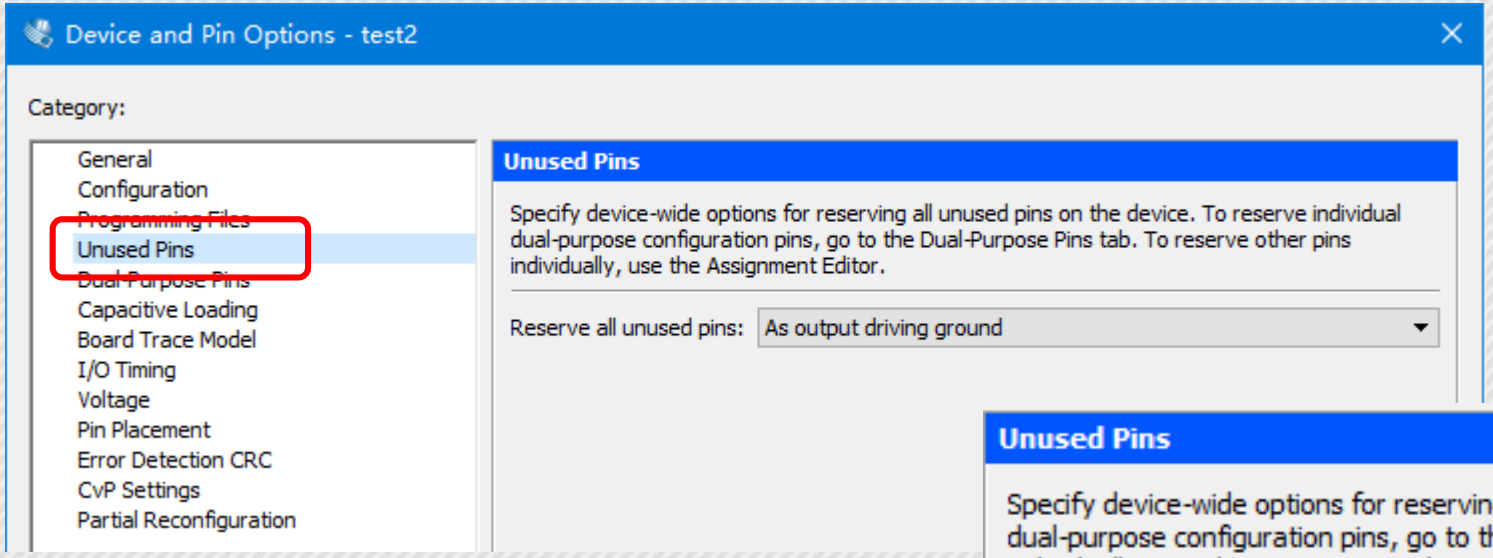


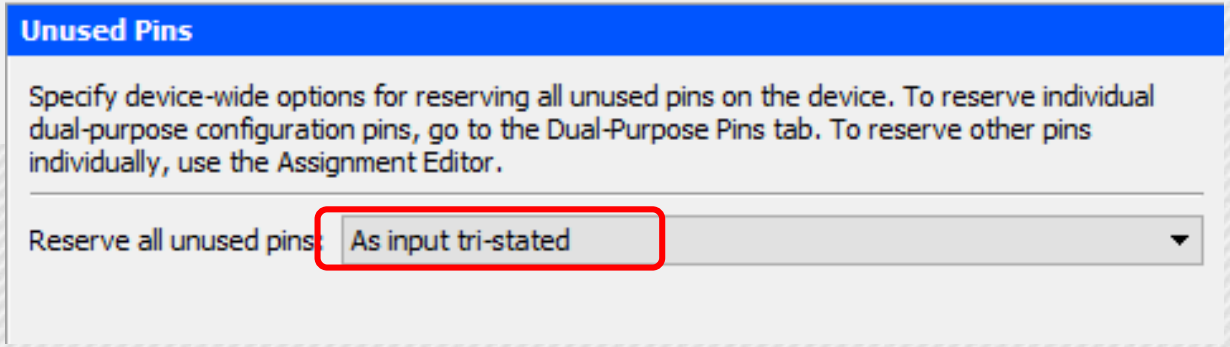


软件使用

设置未用管脚

6. 点击Unused Pins，在右侧选项框中修改为As input tri-stated。

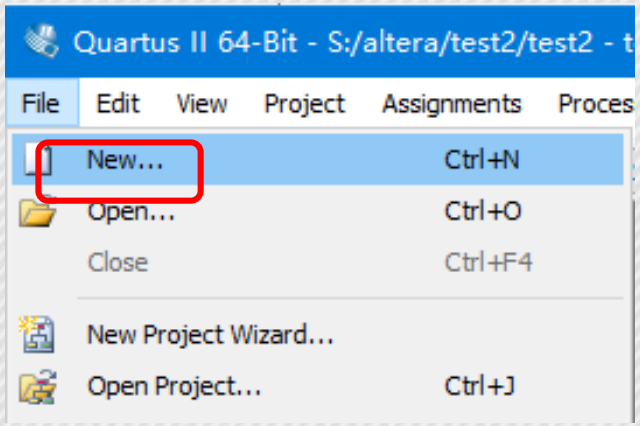


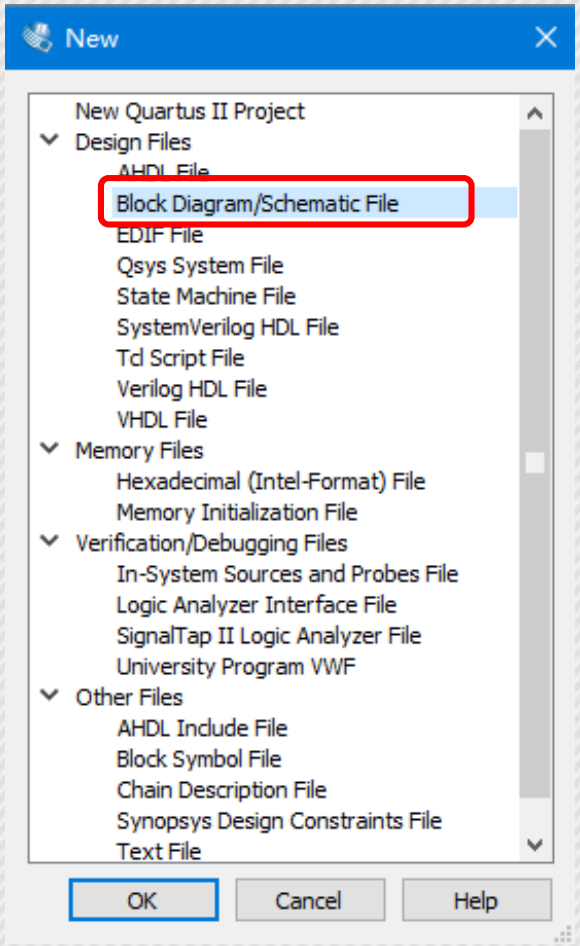


软件使用

创建原理图文件

1.在主界面点击 File-New，在弹出的创建文件界面中，选择Block Diagram/Schematic File。点击 OK 完成创建

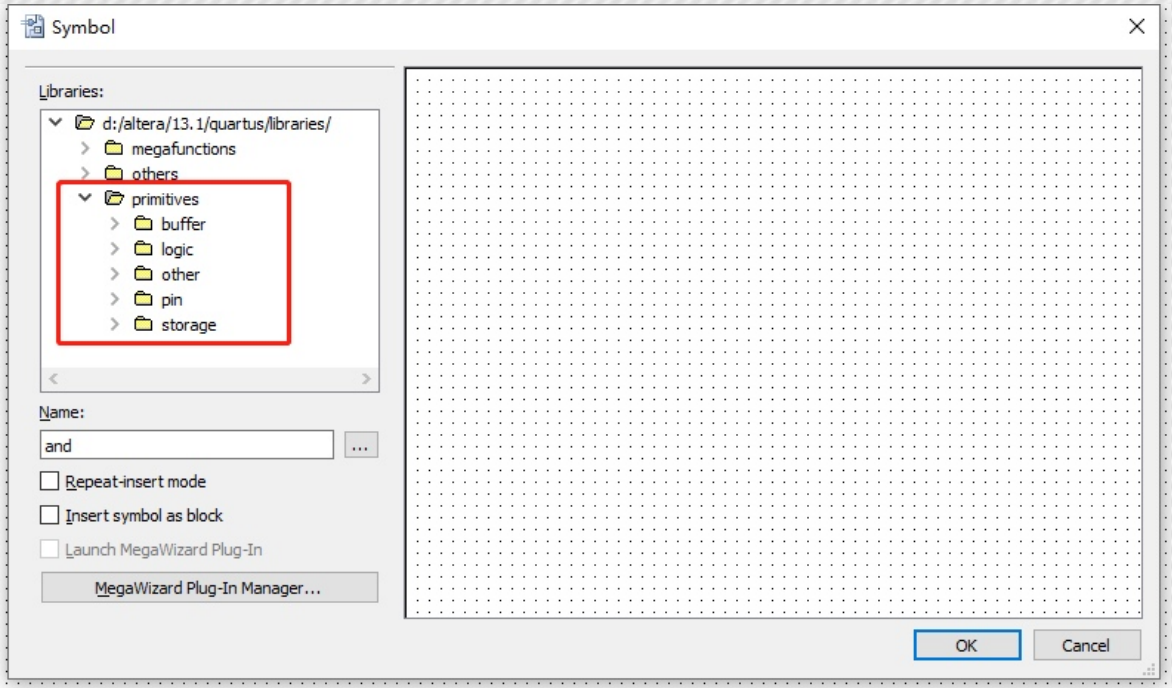


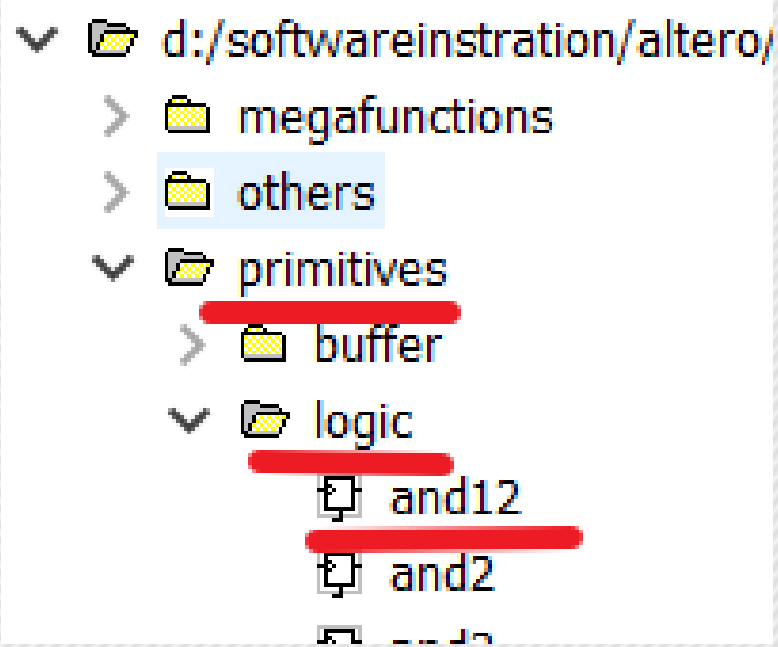


软件使用

创建原理图文件

2.在原理图设计界面中,可以点击 进入Symbol 对话框。Primitives中是基本元器库，在logic 子库中包含与，或，非等门。pin 子库包含输入输出等。根据需要添加元器件

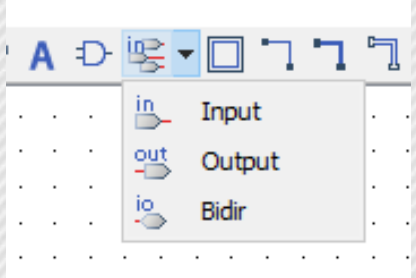


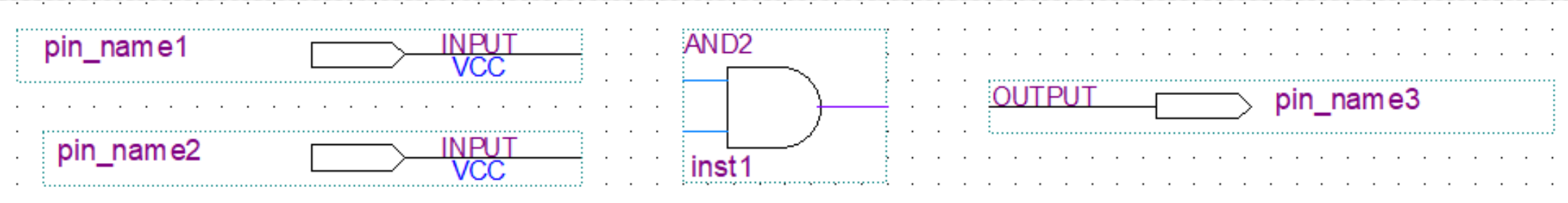


软件使用

创建原理图文件

2.在原理图设计界面中,可以点击 的小三角可以展开端口菜单选择**输入输出端口**器件。 可以用来移动器件， 可以用于缩放视图大小。如下图是添加了一个2输入与门和2个输入端口、以及一个输出端口的图例。

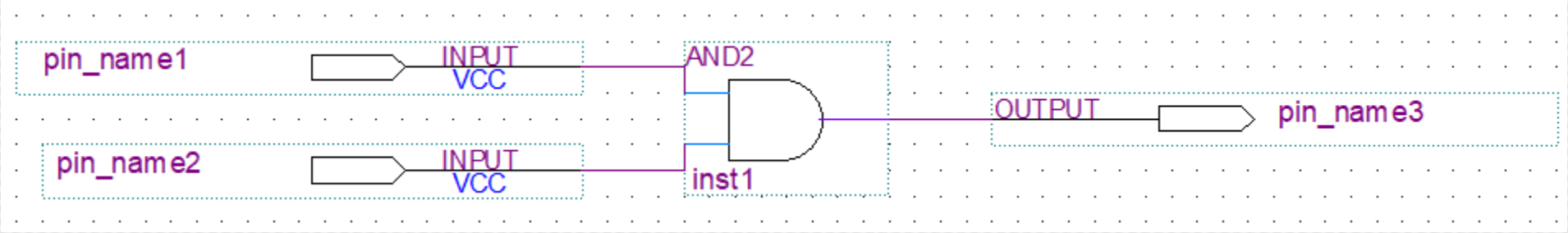




软件使用

创建原理图文件

3.放置元件后，点击 进行连线，根据实验需要绘制原理图



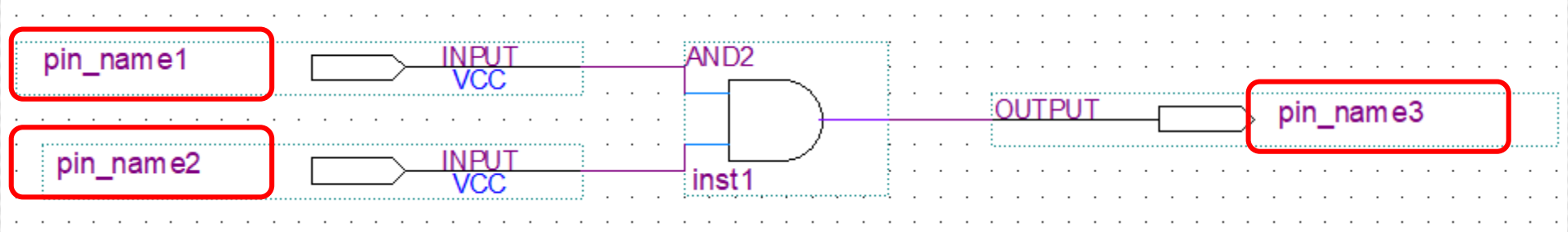
注意事项：

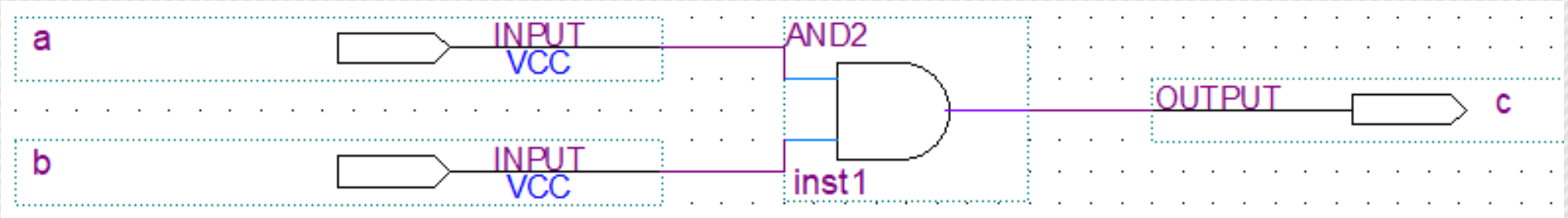
1. 原理图较为复杂，连线的时候需要提前**预留足够的位置**。之后挪动元件位置，相关联的线可能会自动调整位置，从而给后续画图造成不必要的麻烦
2. 确保连线连接正确

软件使用

创建原理图文件

4.双击输入输入元件名字框可以给各输入输出引脚命名

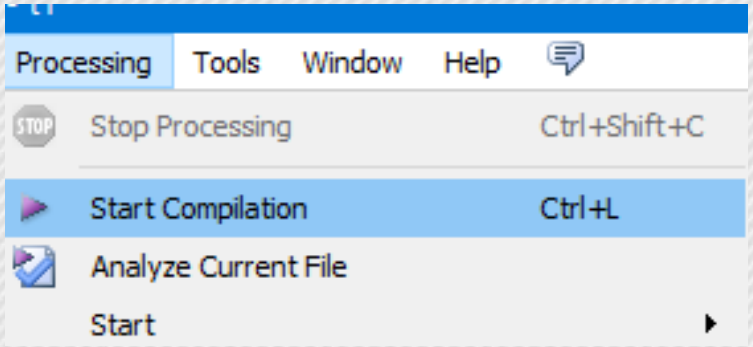


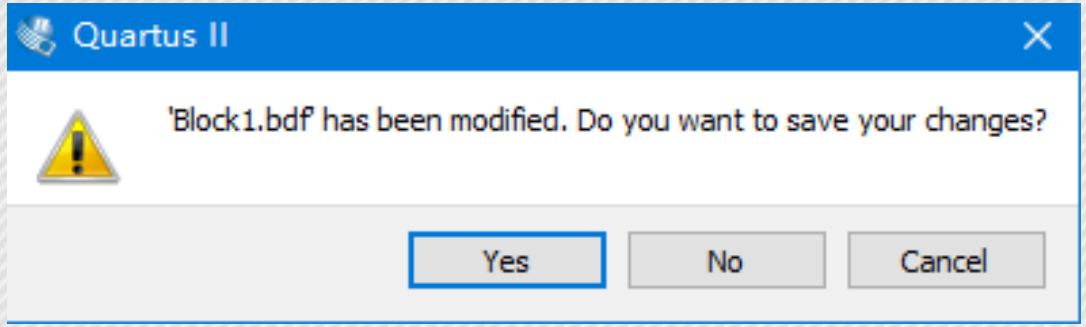


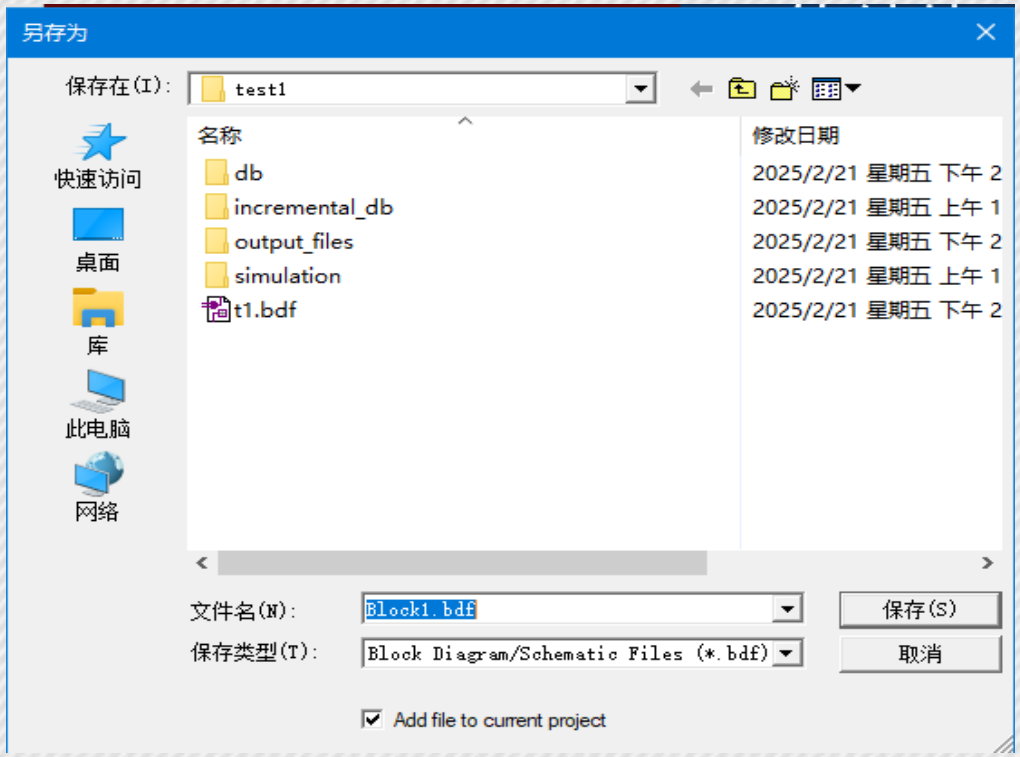
软件使用

创建原理图文件

5.点击Processing-Start Compilation 进行编译，如果有窗口弹出提醒保存则选择Yes，在弹出窗口中选择保存。编译完成后，会给出编译报告。







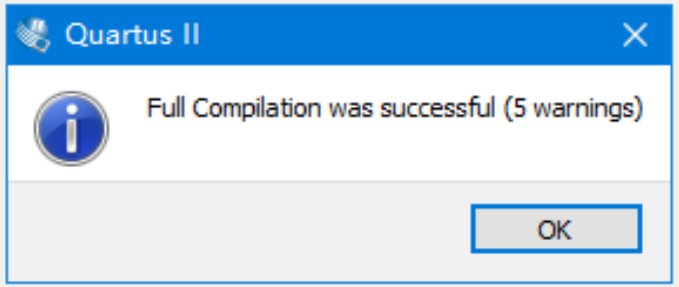
软件使用

创建原理图文件

5.点击Processing-Start Compilation 进行编译，如果有窗口弹出提醒保存则选择Yes，在弹出窗口中选择保存。编译完成后，会给出编译报告。

Flow Summary

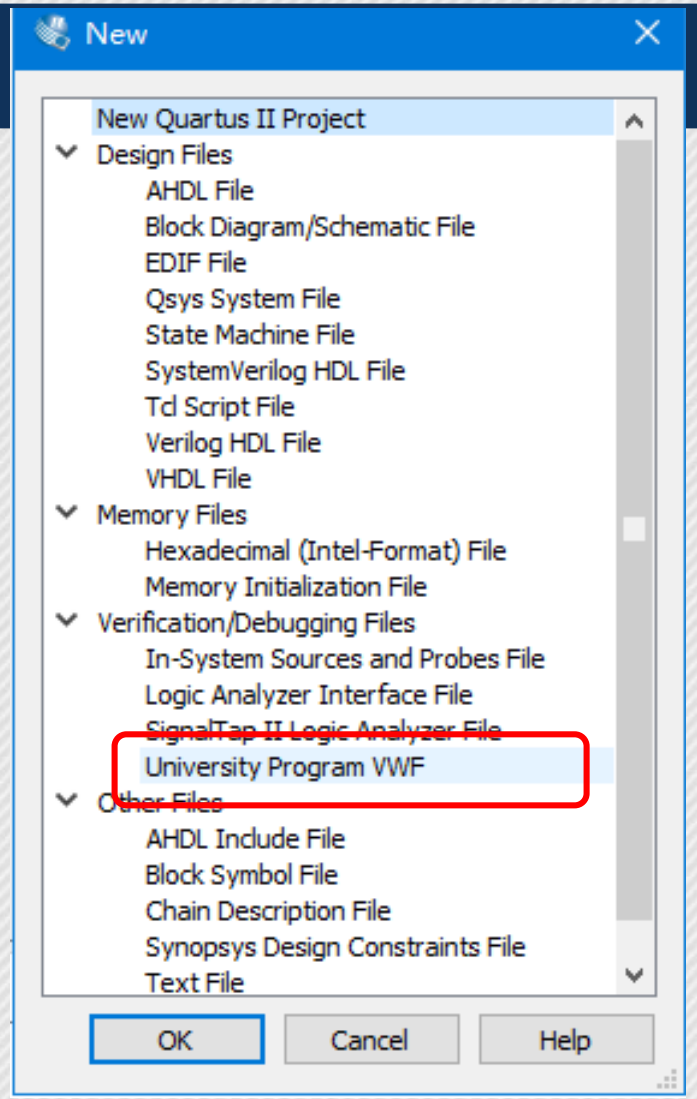
Flow Status	Successful - Fri Feb 21 15:01:10 2025
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Full Version
Revision Name	t1
Top-level Entity Name	t1
Family	Cyclone
Device	EP1C3T144C8
Timing Models	Final
Total logic elements	1 / 2,910 (< 1 %)
Total pins	3 / 104 (3 %)
Total virtual pins	0
Total memory bits	0 / 59,904 (0 %)
Total PLLs	0 / 1 (0 %)



软件使用

时序仿真文件

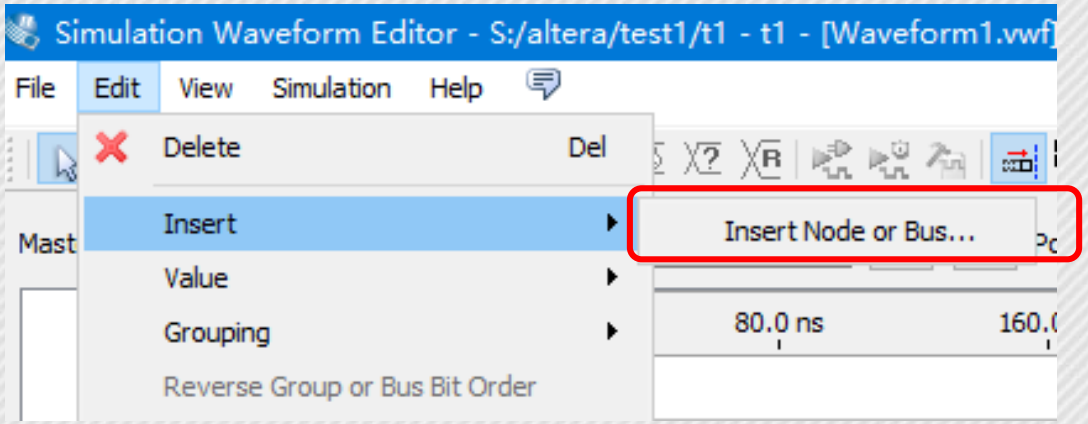
1. 点击File-New 新建用于仿真的波形文件

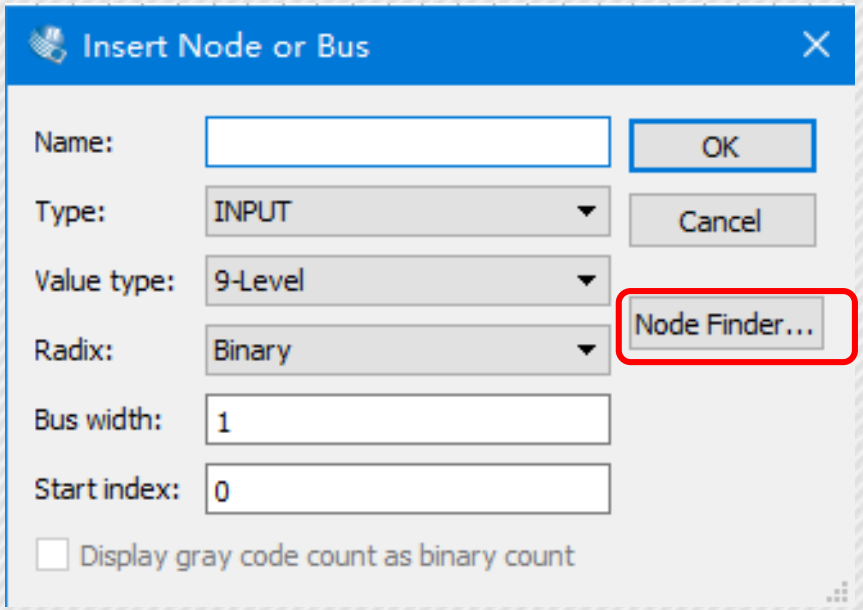


软件使用

时序仿真文件

2.在打开的仿真波形编辑器中，加入之前设计的原理图中的引脚。点击Edit-Insert-Insert Node...弹出右侧窗口后点击Node Finder...

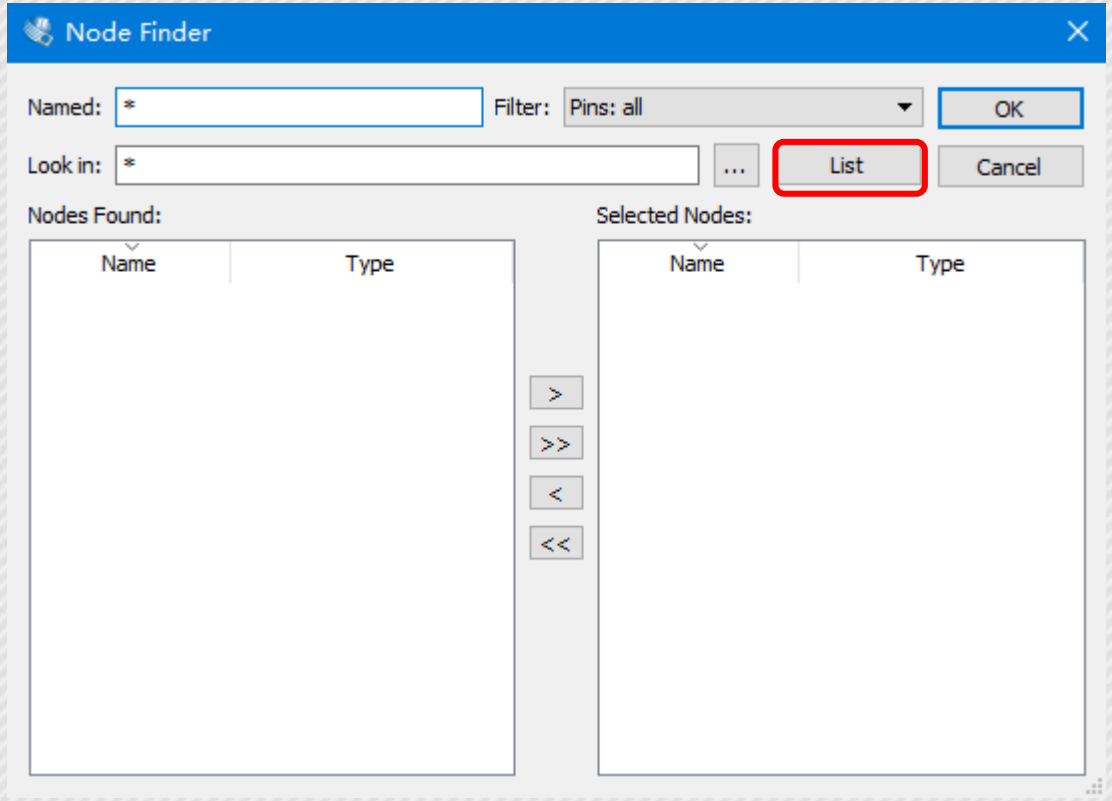


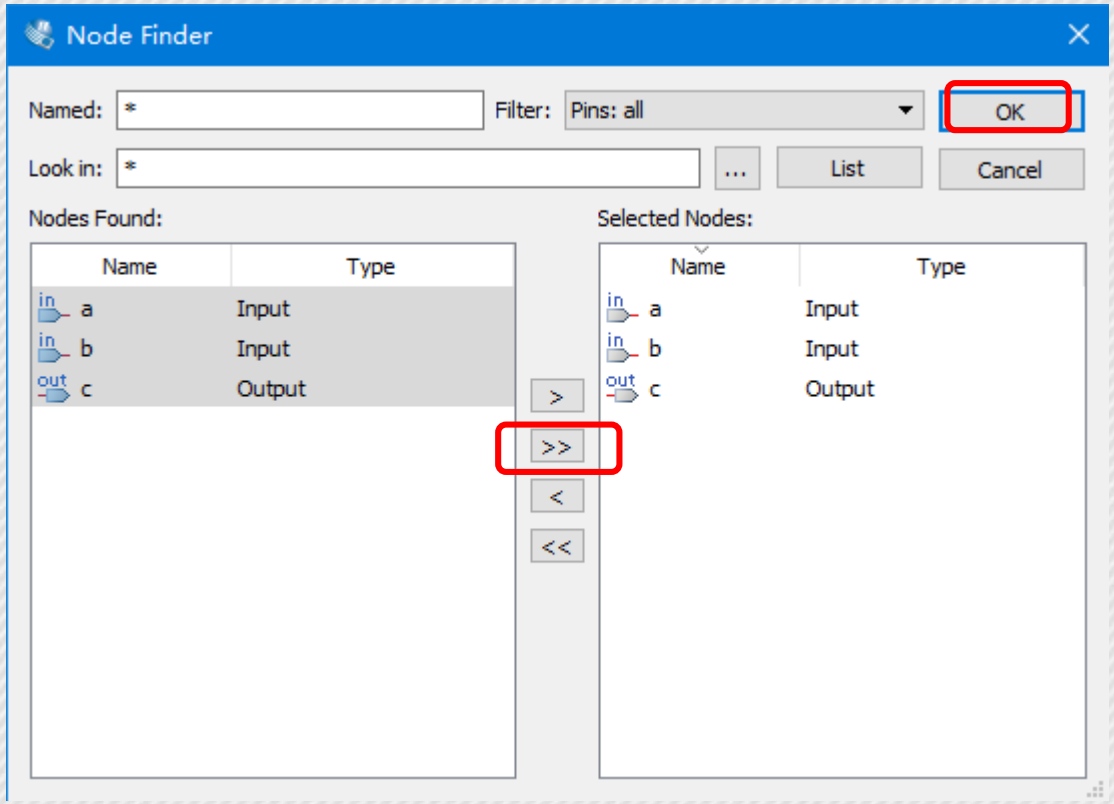


软件使用

时序仿真文件

2. 点击List，左侧窗口出现端口列表后点击>>选中全部端口，点击OK。

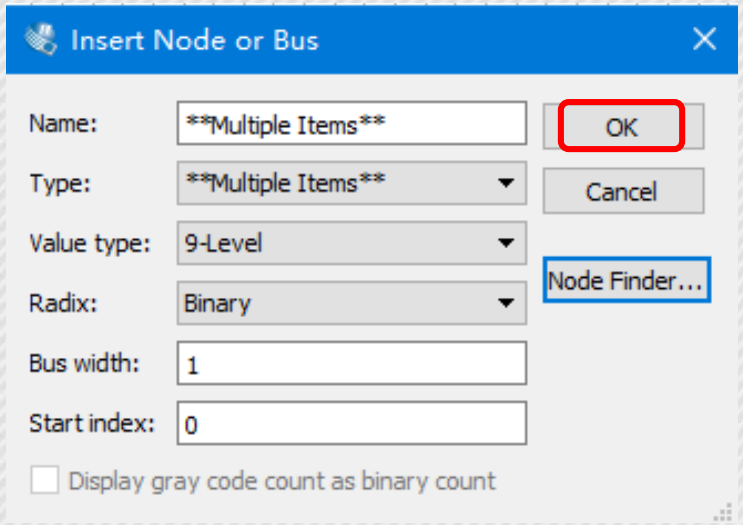


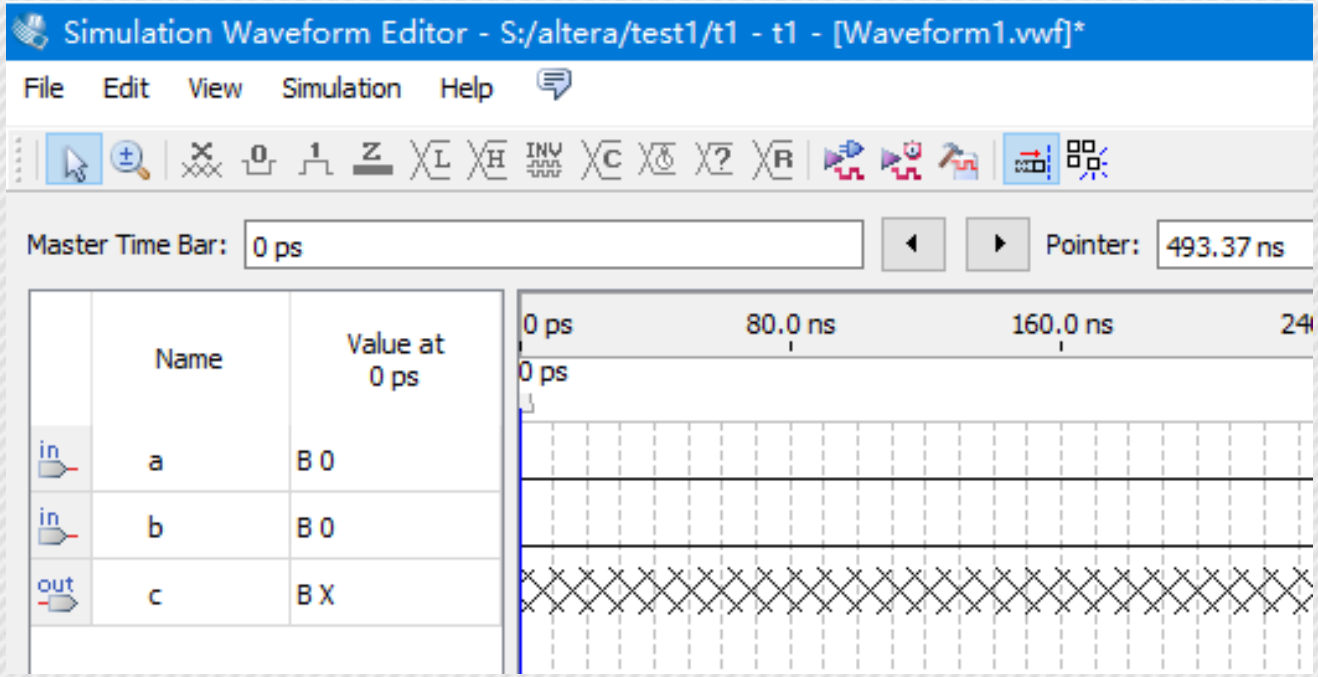


软件使用

时序仿真文件

2. 点击OK，出现右侧图例。

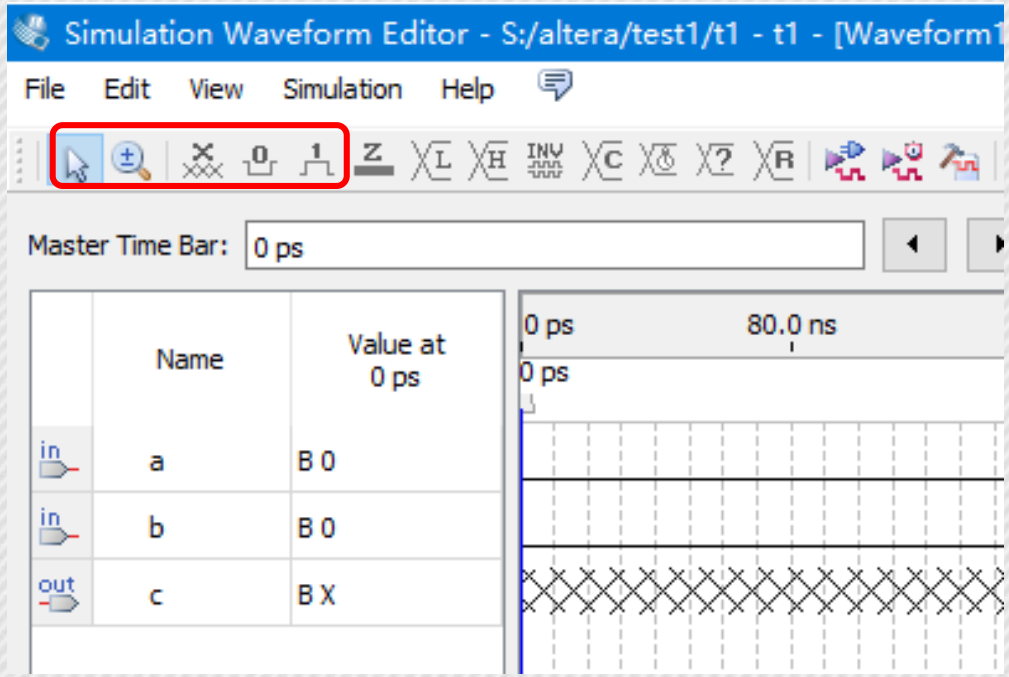


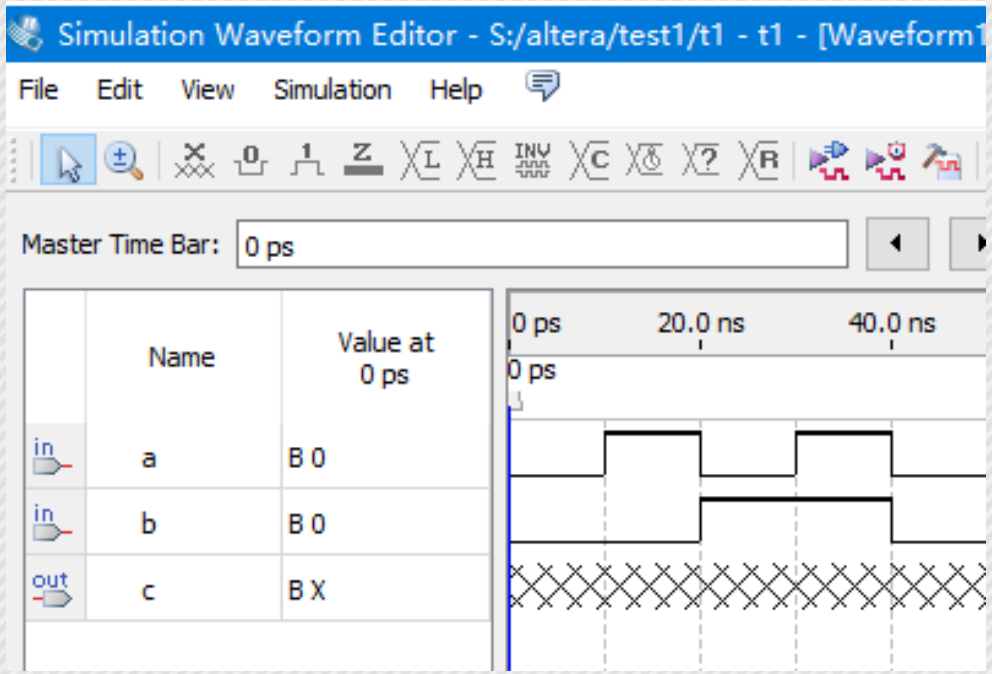


软件使用

时序仿真文件

3. 鼠标选中要设置区域，点击工具栏中的按钮，调整输入信号至需要的状态。原则上，输入信号应该覆盖真值表中的所有输入情况。

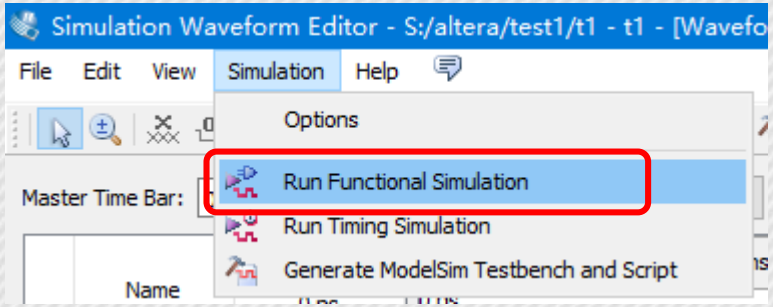


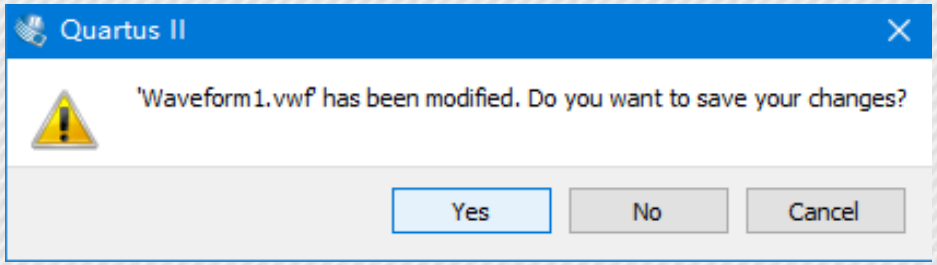


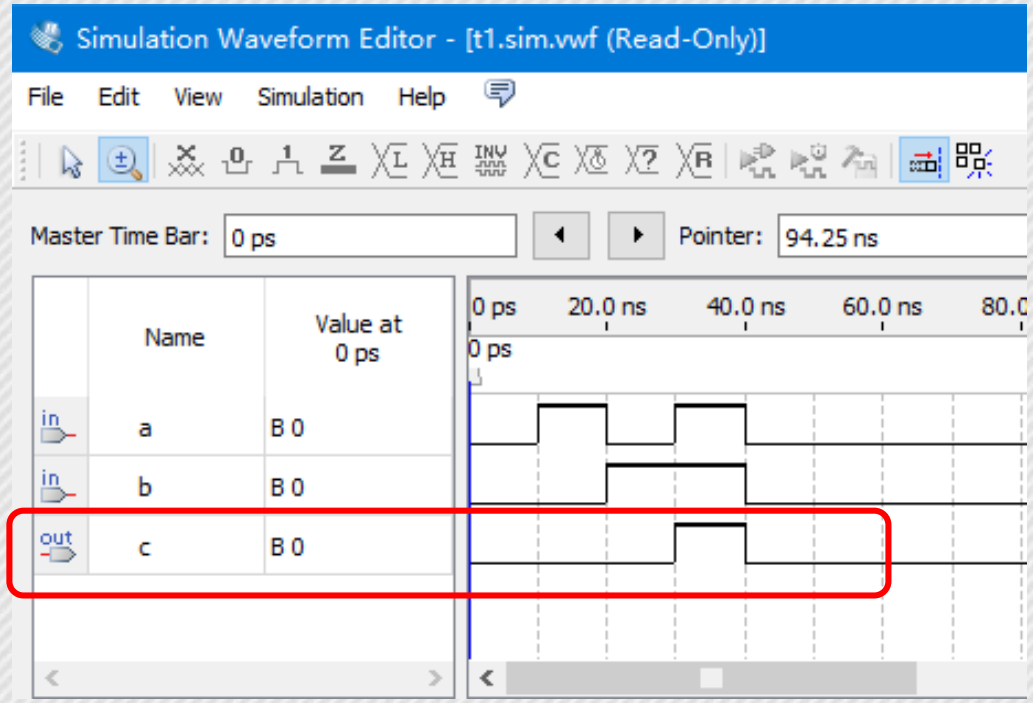
软件使用

时序仿真文件

4. 绘制完激励波形后，点击**Simulation-Run Functional Simulation**，如果有保存提醒则点击**Yes**，进行仿真。仿真完成后，可以看到仿真结果，用来验证设计是否正确。如果**输出跟预期不一致**需要返回检查电路图。





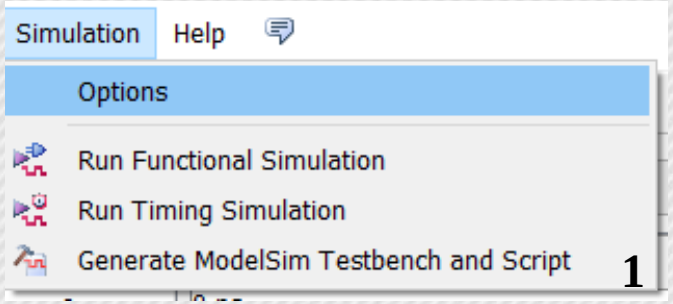


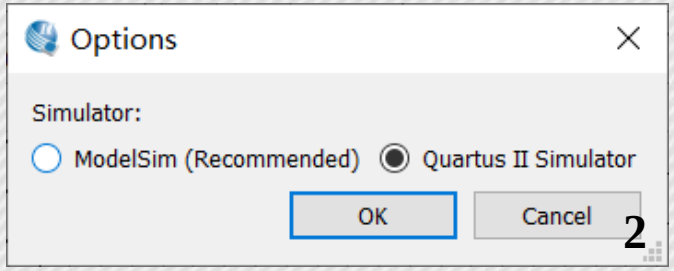
软件使用

时序仿真文件

4. 若提示ModelSim-Altera was not found，如下图所示，可执行以下操作：

```
PID = 10248
**********************************************************************
Running Quartus II 64-Bit EDA Netlist Writer
  Version 13.0.1 Build 232 06/12/2013 Service Pack 1 SJ Full Version
  Processing started: Wed Apr 06 15:46:34 2022
Command: quartus_eda --functional=on --simulation=on --tool=modelsim
_oem --format=verilog one -c one
Generated file one.vo in folder "F:/code/class/4.2digitial/simulatio
n/modelsim/" for EDA simulation tool
Quartus II 64-Bit EDA Netlist Writer was successful. 0 errors, 0 war
nings
  Peak virtual memory: 4506 megabytes
  Processing ended: Wed Apr 06 15:46:34 2022
  Elapsed time: 00:00:00
  Total CPU time (on all processors): 00:00:01
**********************************************************************
ModelSim-Altera was not found. Please install ModelSim-Altera which
is included with the Quartus II installer, or use the Quartus II Sim
ulator instead by selecting "Simulation > Options > Quartus II Simul
ator"
```



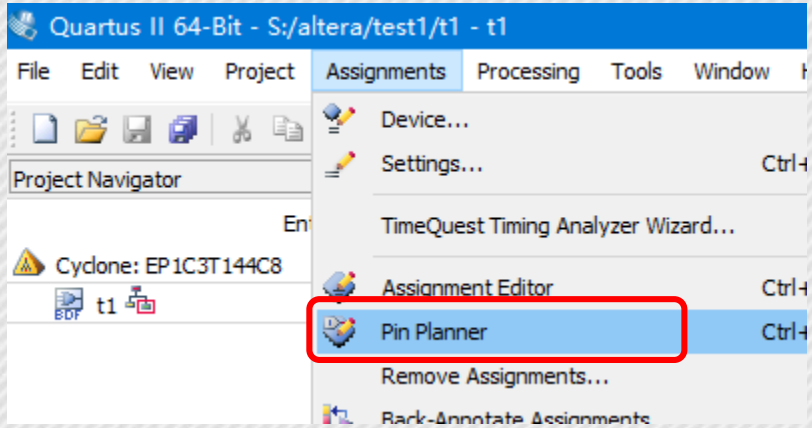


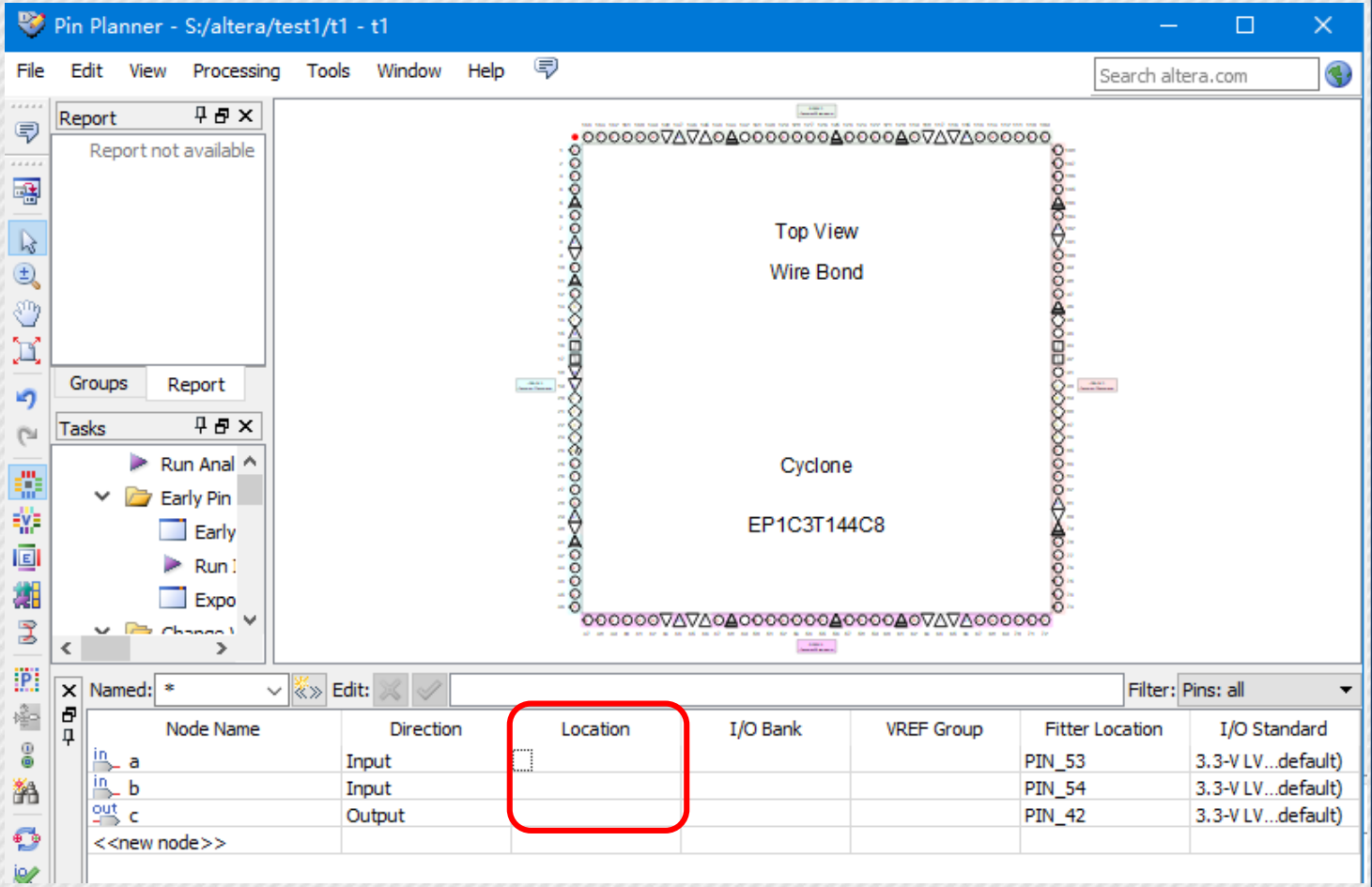
选择Quartus II Simulator，并选择OK

软件使用

添加管脚约束，生成 bit 流并下载到实验板

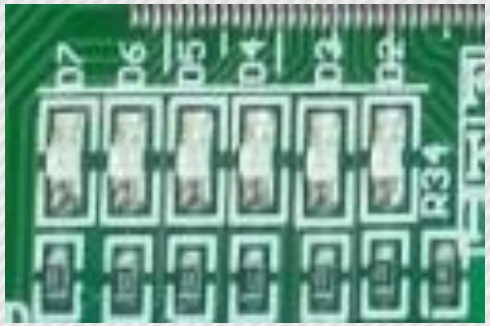
1. 点击 **Assignments-Pin Planner** 添加管脚约束，在 **Location** 中点击**小虚线方块**为对应的输入输出设置管脚。





软件使用

管脚约束示例



P42 P47 P49 P52 P55 P57

P35 P36 P38 P37 P39 P40 P41 P48 P50 P51 P53 P54

LED灯都是低有效，即输出为0时点亮。

左侧开关等效上1下0，右侧开关等效上0下1。

下面的设置将端口a和右侧拨码开关3绑定，端口b和右侧拨码开关4绑定，端口c和LED灯D7绑定。

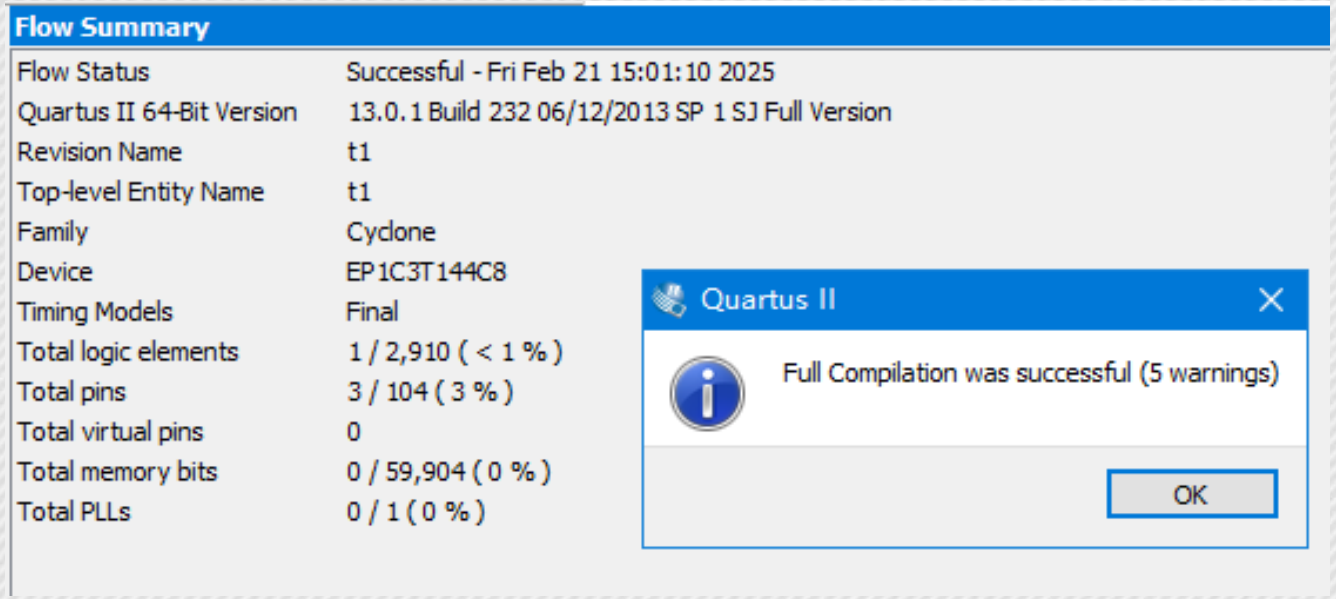
Named: * Edit: Filter: P

Node Name	Direction	Location	I/O Bank	VREF Group	Fitter Location
in a	Input	PIN_50	4	B4_N2	PIN_53
in b	Input	PIN_51	4	B4_N2	PIN_54
out c	Output	PIN_42	4	B4_N2	PIN_42
<<new node>>					

软件使用

重新编译生成 bit 流

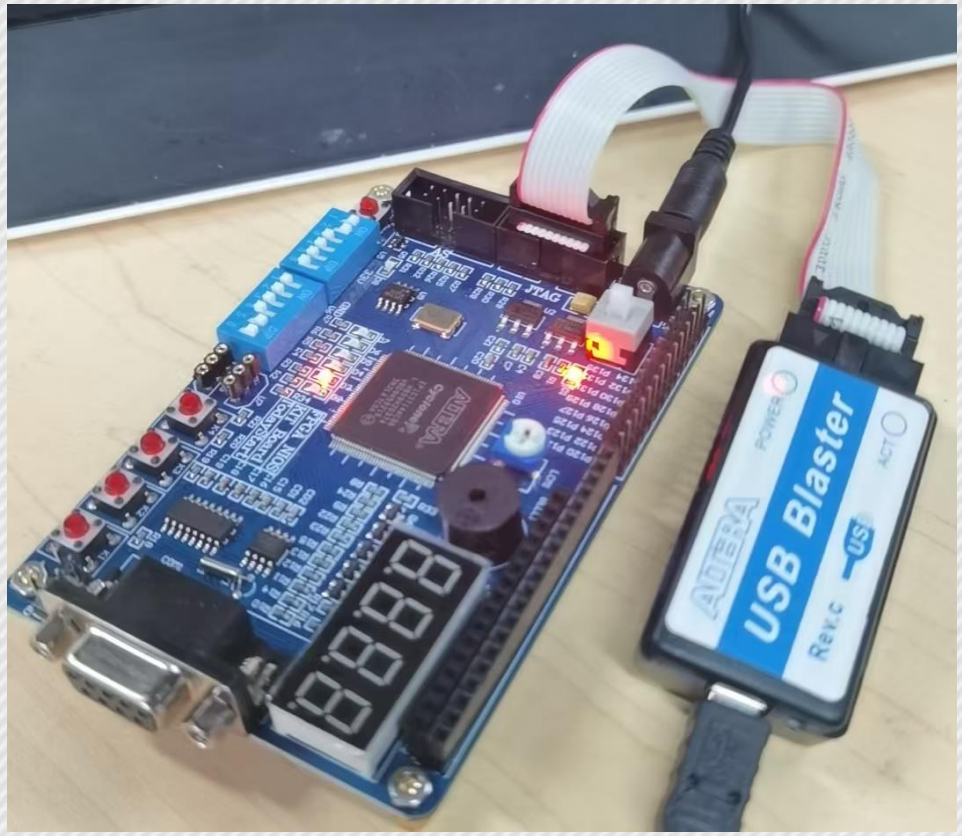
2. 点击Processing-Start Compilation 进行编译，如果有窗口弹出提醒保存则选择Yes，在弹出窗口中选择保存。编译完成后，会给出编译报告。



软件使用

连接实验板

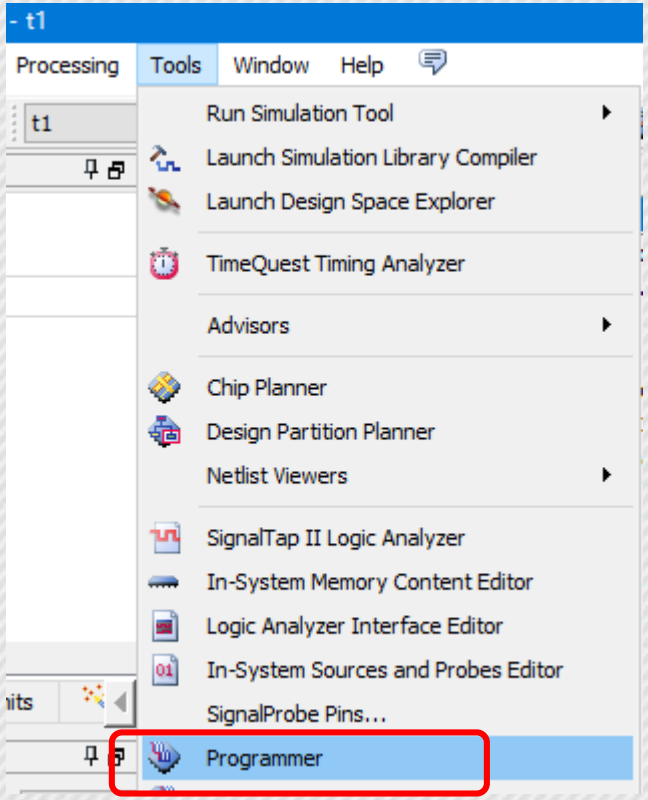
3. 为实验板供电，将USB Blaster连接至电脑的USB口，另一端连接到板子的JTAG口。

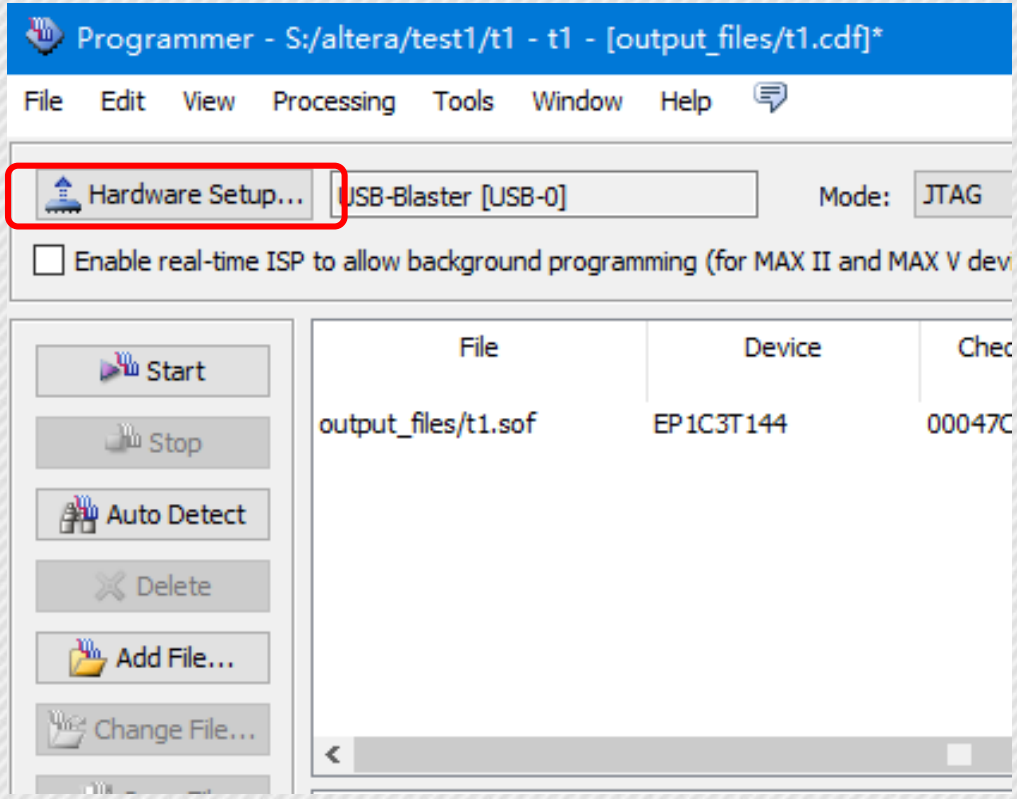


软件使用

下载到实验板

3. 点击 Tools-Programmer，在Programmer中点击Hardware Setup。

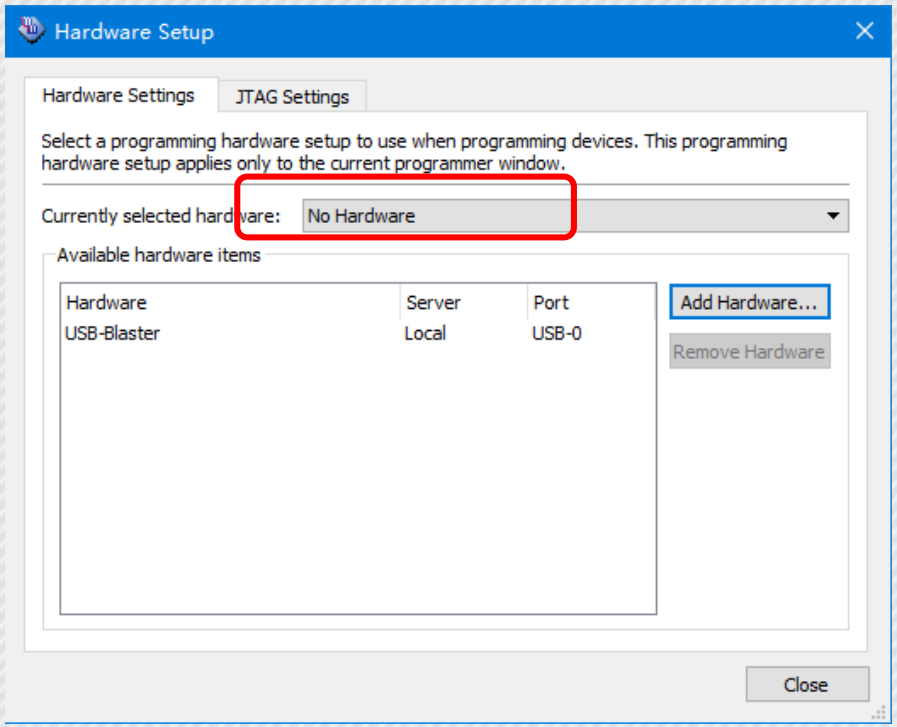


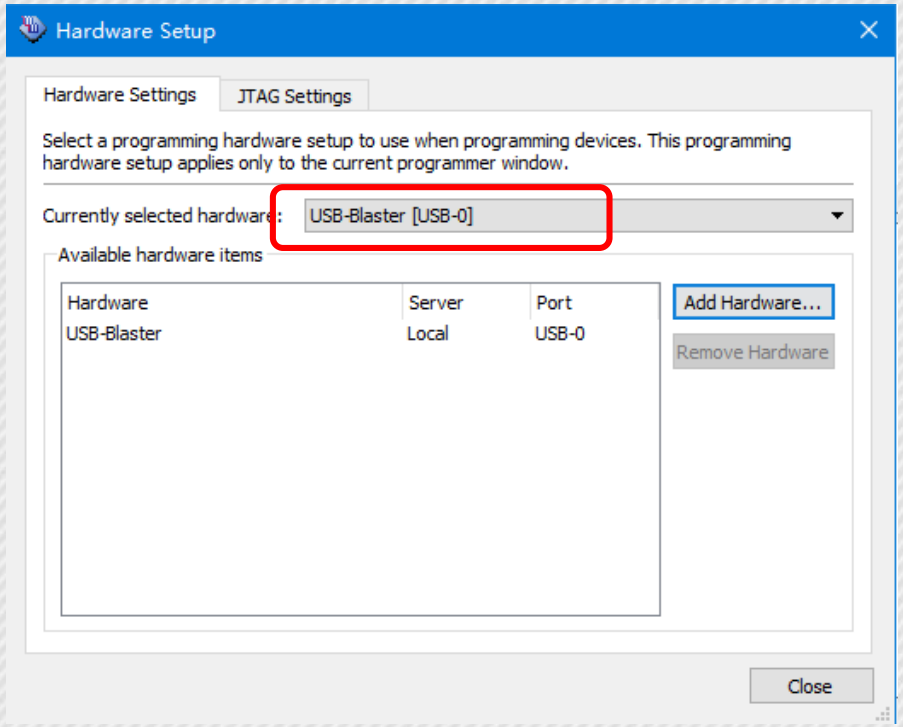


软件使用

连接设备

4. 选择USB-Blaster。



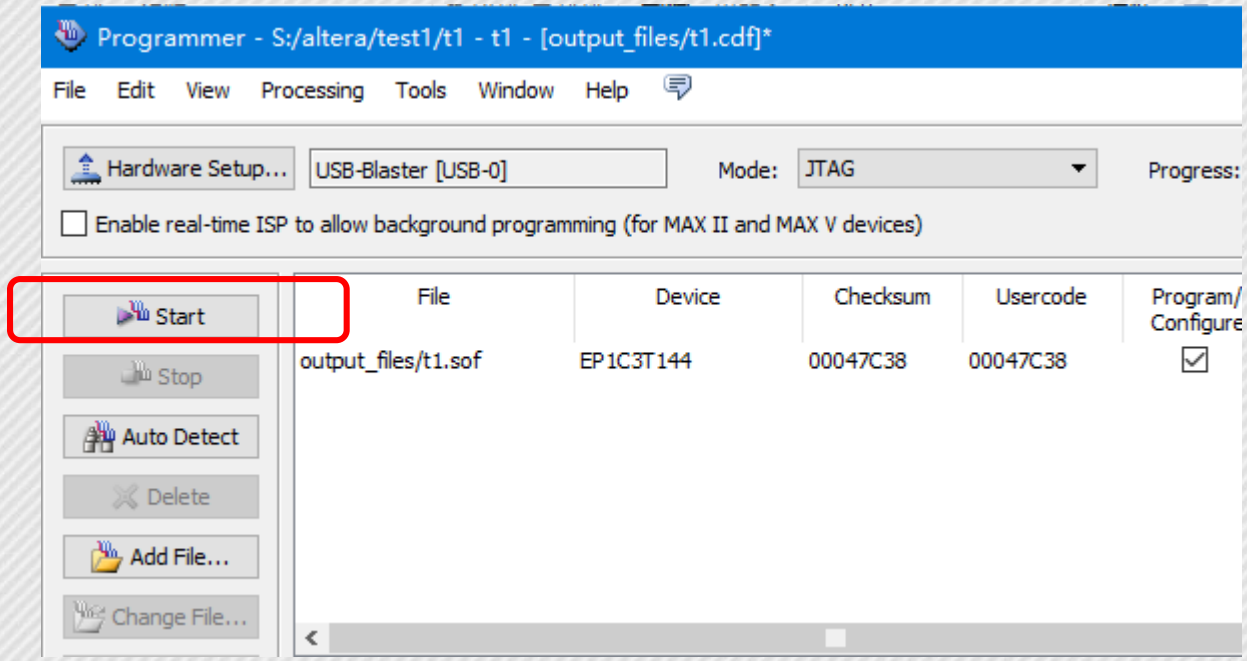


备注：若不能正确传到开发板上，请检查驱动是否正确安装—USB Blaster

软件使用

下载程序

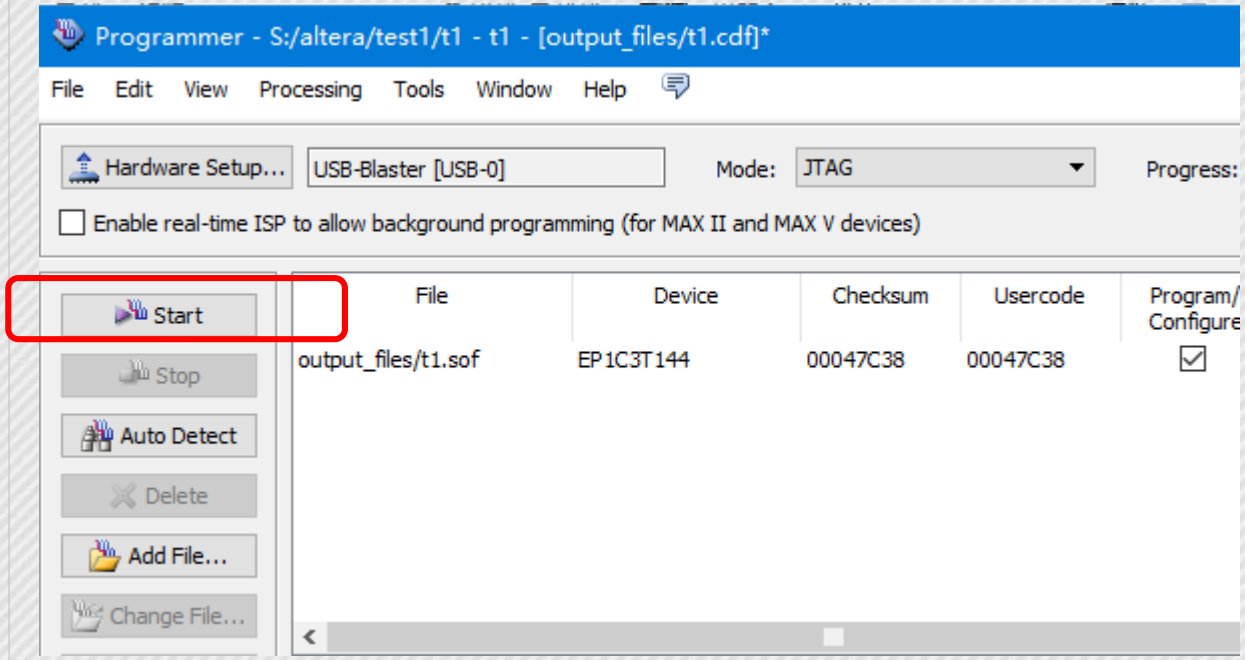
5. 点击Start往实验板下载程序，成功可以看到右上出现 Progress: 100% (Successful) ，这时就可以在实验板上拨动拨码开关观察实验现象了。



软件使用

下载程序

3. 点击Start往实验板下载程序，成功可以看到右上出现 Progress: 100% (Successful) ，这时就可以在实验板上拨动拨码开关观察实验现象了。



观察实验结果时需要注意：左侧开关等效上1下0，右侧开关等效上0下1。LED灯都是低有效，即输出为0时点亮。

实验一

控制你的数码管显示对应输入数字

实验要求：用实验板上**4位拨码开关**作为**8421BCD码**的输入，设计**七段数码管**显示驱动电路，在数码管上显示8421BCD码编码的十进制数字。

实验目标：（1）学会Quartus II软件的使用，利用其进行原理图的设计。

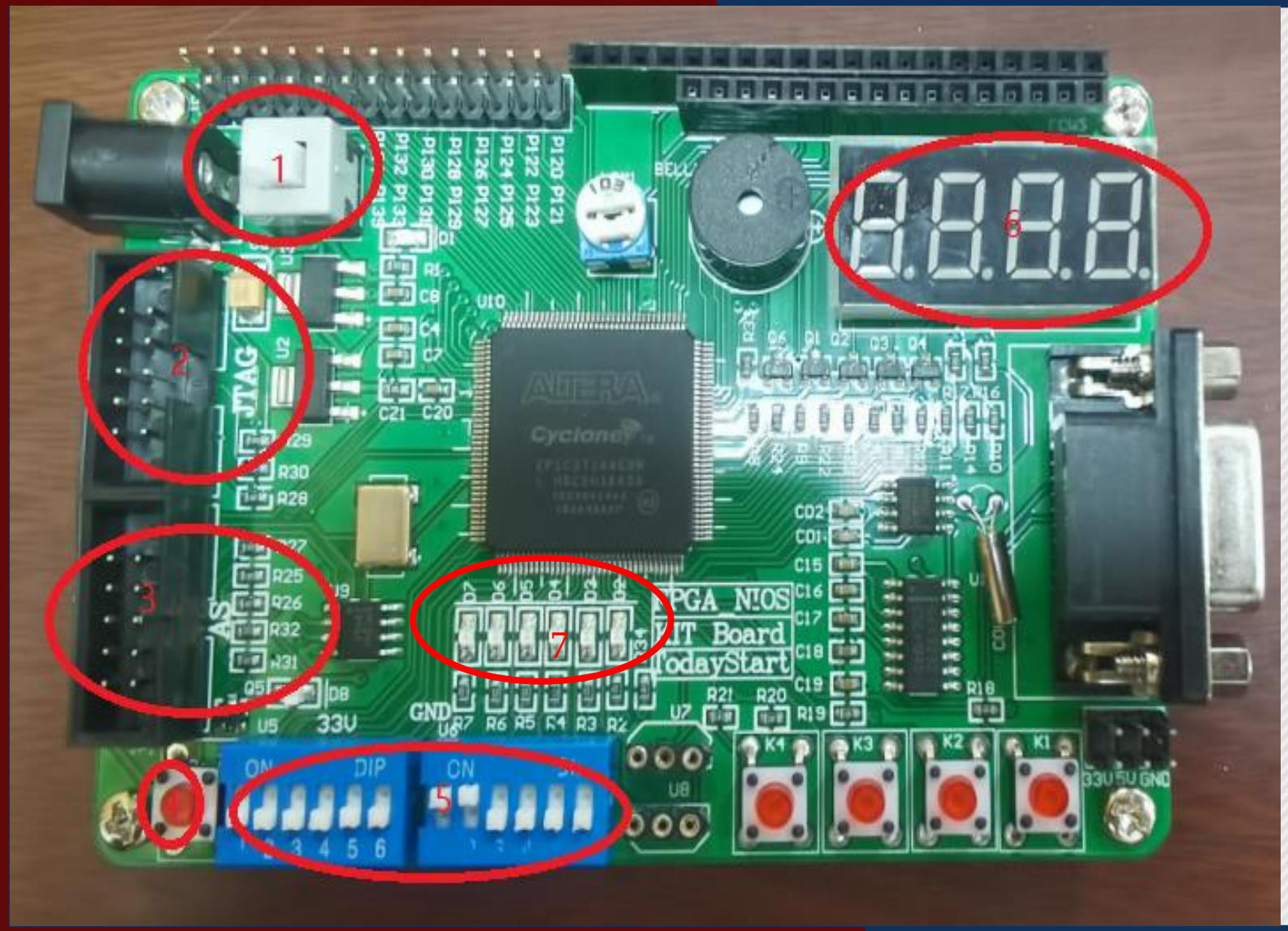
（2）加深对课堂学习的组合逻辑电路的认识，学会设计基础的组合逻辑电路。

（3）增强实际动手能力，将设计电路运行在开发板上。

步骤

1. 先完成将8421BCD码的输入转换为七段数码管显示的组合电路设计。
2. 创建工程，选择需要的器件构造电路图。
3. 再按照前面的步骤完成仿真，检查自己的电路是否正确。
4. 仿真结果正确再下载到实验板上进行测试。
5. 编写实验报告，报告需要包含电路设计过程、电路图、仿真图、结果图和实验分析。

FPGA开发板示意图



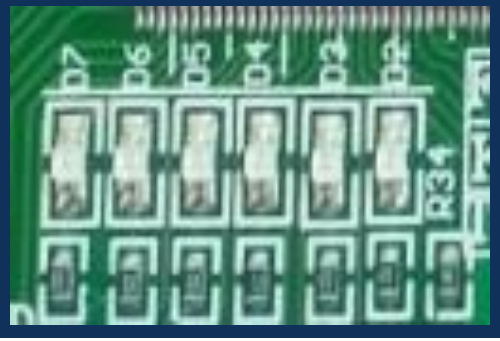
1. **电源开关**
2. **JTAG接口**：电可擦，断电之后程序自动删除。数据写在FPGA主芯片中
3. **AS接口**：电可擦，断电后程序仍然保存。数据写在EPCS4芯片当中。
4. **重置键**：还原原始程序，为FPGA芯片重新分配数据
5. **拨码开关**：可以作为输入
6. **显示器**：可以发光显示数字。下面的原点为小数点，默认会亮起
7. **LED灯**：可以作为输出

拨码开关与LED灯

- 此次实验，输入采用的是**拨码开关**作为输入信号。由于只需要控制1个数码管显示即可，因此只需要四位开关作为输入。
- 引脚与开关对应关系，引脚**P50、P51、P53、P54**对应器件上为右上图中**右边拨码开关的3、4、5、6位开关**。
- 左边开关打开后，等效输入为1，而**右边开关打开后，等效输入为0**。
- 注意：一路为低电平，一路为高电平，可以通过拨动拨码开关实现输入高或者输入低电平。

P35 P36 P38 P37 P39 P40　P41 P48 P50 P51 P53 P54

左侧开关等效上1下0，右侧开关等效上0下1。

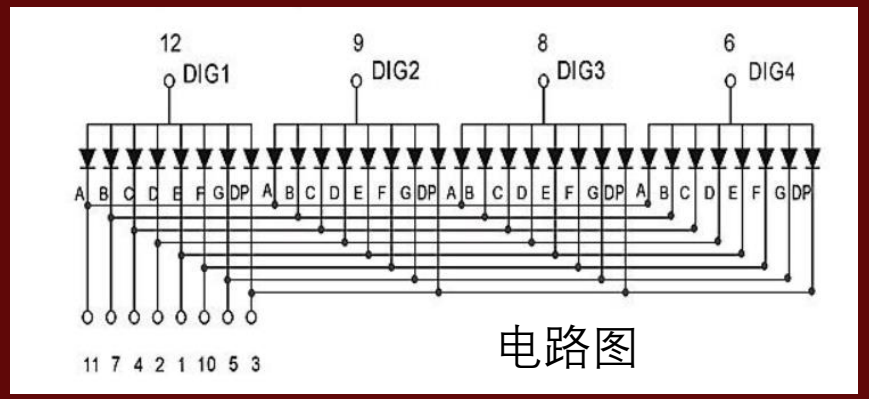


P42 P47 P49 P52 P55 P57

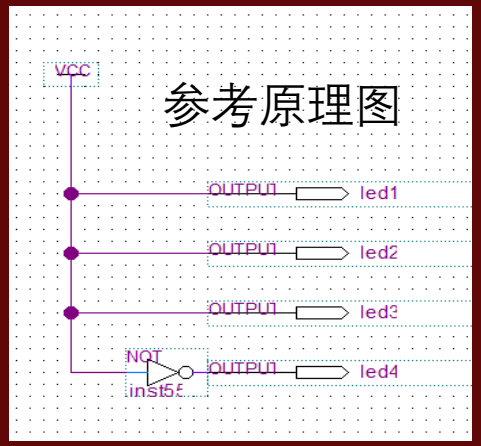
LED灯都是低有效，即输出为0时点亮。

四位共阳极数码管

对应板上器件



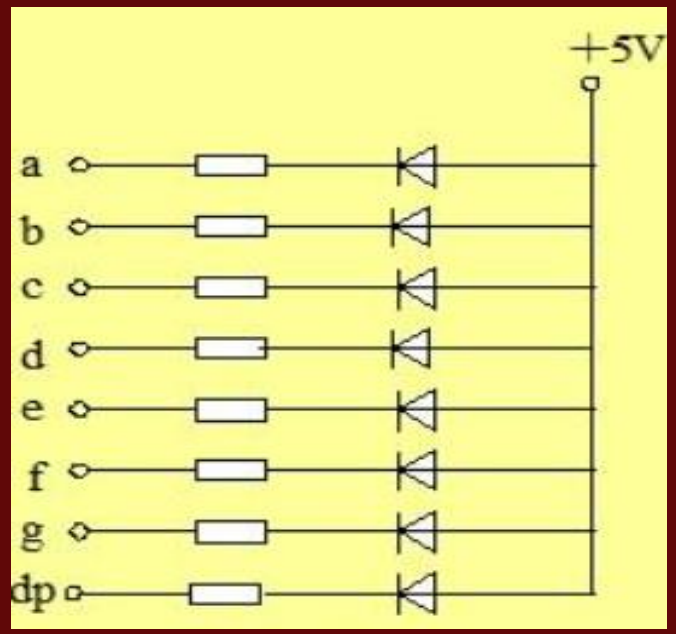
电路图



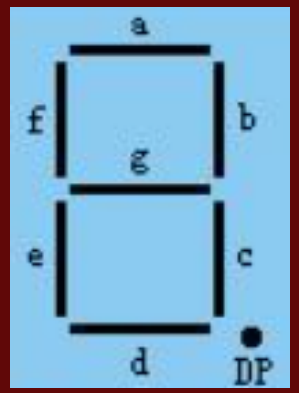
参考原理图

- 引脚P91,P85,P84,P83分别对应控制4个LED灯，按照从左到右的顺序，由于其为共阳极数码管，则为低电平有效。
- 举例来说，如果P91引脚输入信号为0，其他3个引脚输入信号为1，则4个数码管只有最左边的会有效（即会亮起来），其他的均不工作；同理，P83引脚输入信号为0，其他3个引脚输入信号为1，是最右边的数码管有效。
- 想要使用哪一个数码管有效，需要将其输入电平置0。在参考电路图中，即控制LED4发光（即最右侧的数码管），其他均不发光。

七段数码管显示

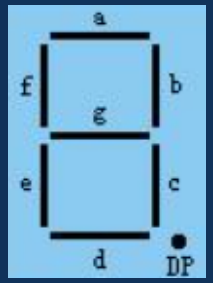


- 每一个**数码管**，都是一个**七段译码器**，从原理图可以看出，8个引脚分别对应A~G，和DP（小数点位，此次实验用不上），此数码管为**低电平有效**，即你想使该端发光，那么输入信号应当是0。
- 比如你想让数码管显示1，那么需要使B,C对应段发光。此时，B,C引脚输入应当为0，而其他引脚输入应当为0。



位控制管脚	FPGA对应管脚	段码控制	FPGA对应	段码控制	FPGA对应
LED1	P83	LEDA	P74	LEDE	P75
LED2	P84	LEDB	P77	LEDF	P82
LED3	P85	LEDC	P78	LEDG	P73
LED4	P91	LEDD	P79	LEDP	P76

BCD-七段数码管显示



	w	x	y	z	a	b	c	d	e	f	g
0	0	0	0	0	0	0	0	0	0	0	1
1	0	0	0	1	1	0	0	1	1	1	1
2	0	0	1	0	0	0	1	0	0	1	0
3	0	0	1	1	0	0	0	0	1	1	0
4	0	1	0	0	1	0	0	1	1	0	0
5	0	1	0	1	0	1	0	0	1	0	0
6	0	1	1	0	0	1	0	0	0	0	0
7	0	1	1	1	0	0	0	1	1	1	1
8	1	0	0	0	0	0	0	0	0	0	0
9	1	0	0	1	0	0	0	0	1	0	0
	……	……	……	……	d	d	d	d	d	d	d

BCD-七段数码管设计

$A = xy'z' + w'x'y'z$

$B = xy'z + xyz'$

$C = w'x'yz'$

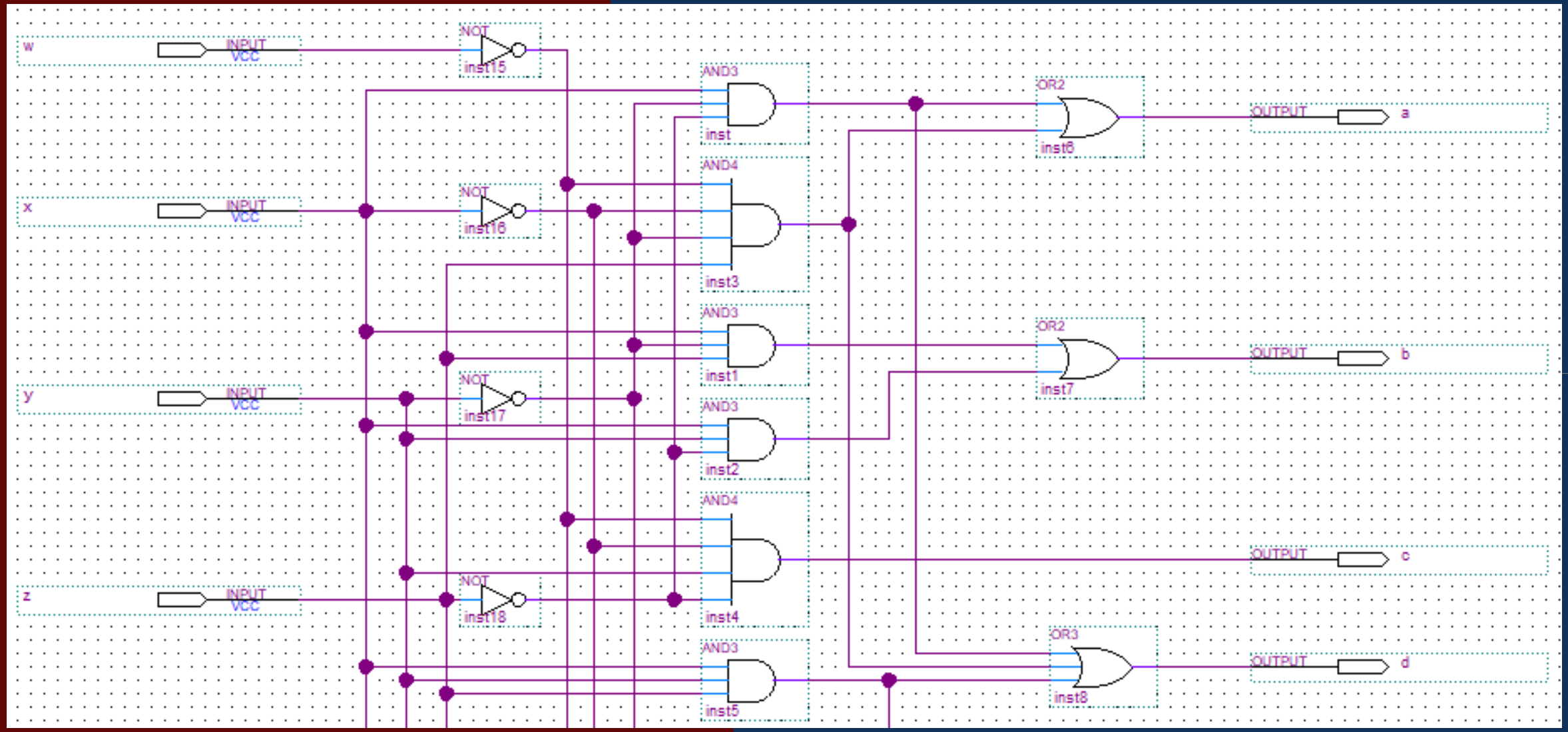
$D = xy'z' + w'x'y'z + xyz$

$E = z + xy'$

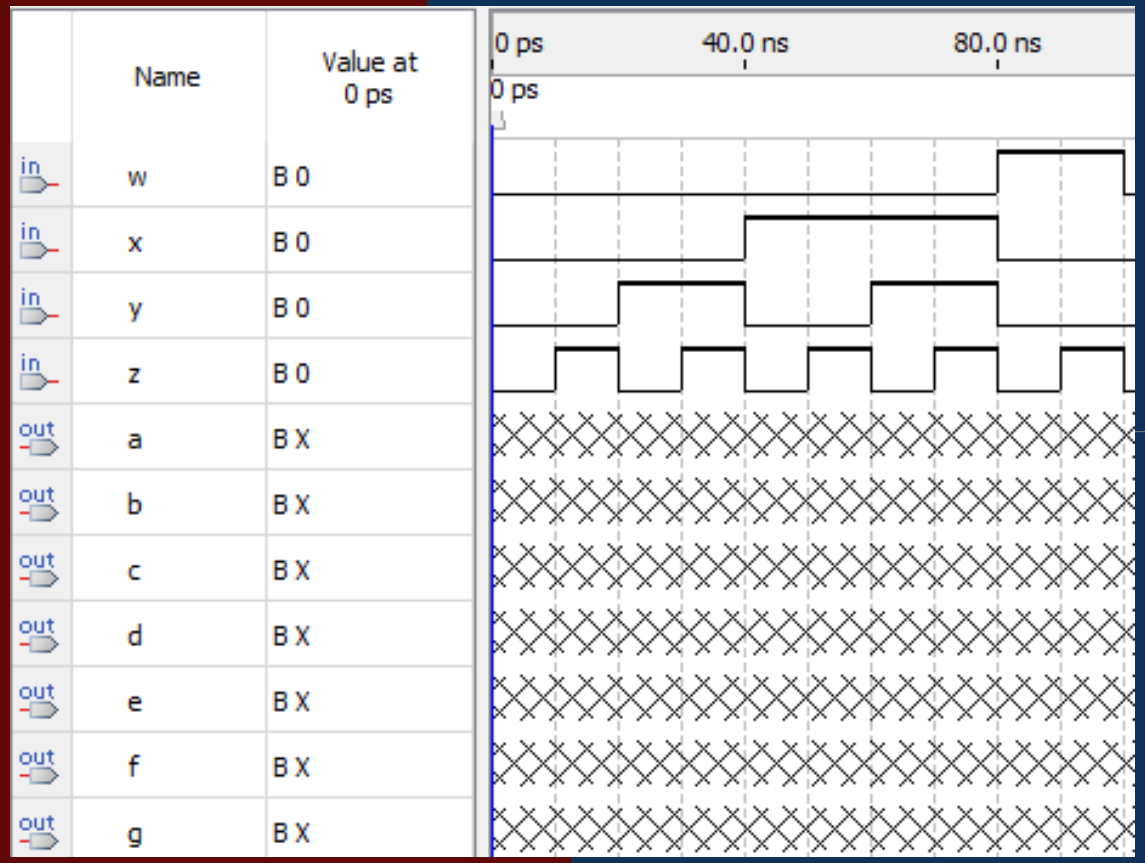
$F = yz + w'x'z + w'x'y$

$G = w'x'y' + xyz$

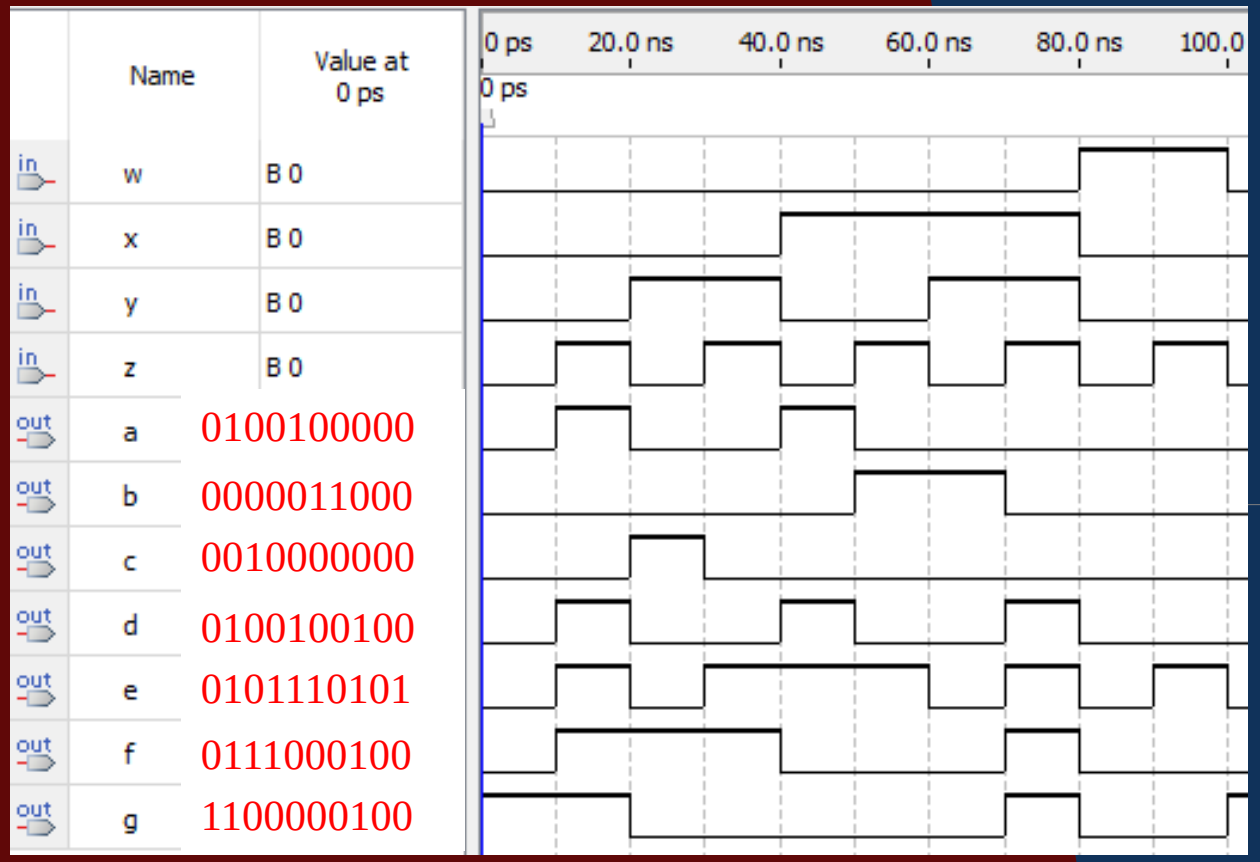
部分电路图


w
INPUT
VCC
NOT
inst15
AND3
inst
OR2
inst6
OUTPUT
a
x
INPUT
VCC
NOT
inst16
AND4
inst3
AND3
inst1
OR2
inst7
OUTPUT
b
y
INPUT
VCC
NOT
inst17
AND3
inst2
AND4
inst4
OUTPUT
c
z
INPUT
VCC
NOT
inst18
AND3
inst5
OR3
inst8
OUTPUT
d

仿真示例


Name
Value at 0 ps
0 ps
40.0 ns
80.0 ns
0 ps
in w B 0
in x B 0
in y B 0
in z B 0
out a B X
out b B X
out c B X
out d B X
out e B X
out f B X
out g B X

仿真示例

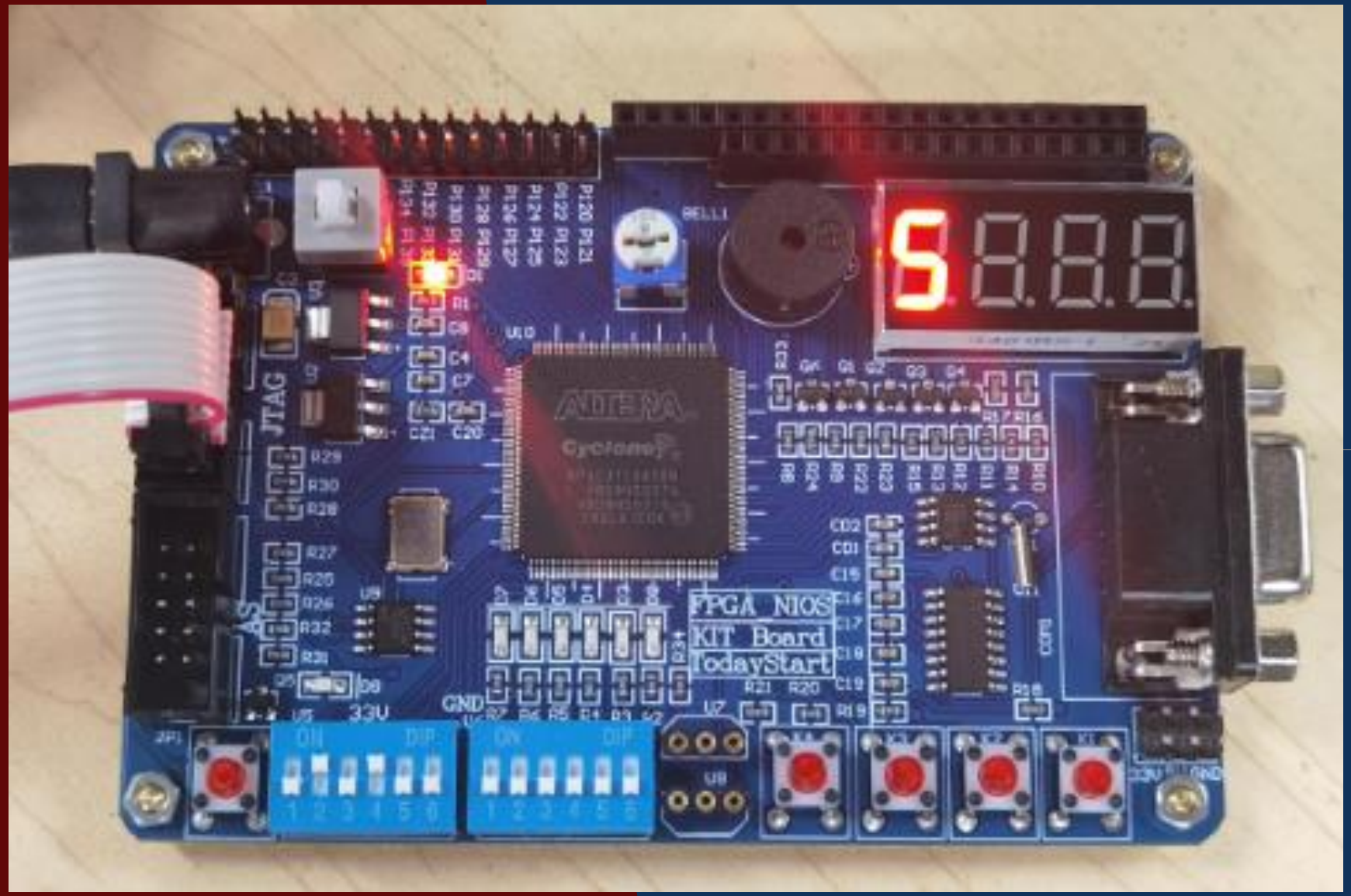


	w	x	y	z	a	b	c	d	e	f	g
0	0	0	0	0	0	0	0	0	0	0	1
1	0	0	0	1	1	0	0	1	1	1	1
2	0	0	1	0	0	0	1	0	0	1	0
3	0	0	1	1	0	0	0	0	1	1	0
4	0	1	0	0	1	0	0	1	1	0	0
5	0	1	0	1	0	1	0	0	1	0	0
6	0	1	1	0	0	1	0	0	0	0	0
7	0	1	1	1	0	0	0	1	1	1	1
8	1	0	0	0	0	0	0	0	0	0	0
9	1	0	0	1	0	0	0	0	1	0	0
	…	…	…	…	d	d	d	d	d	d	d

管脚约束示例

拨码开关	管脚	拨码开关	管脚
1	P35	4	P37
2	P36	5	P39
3	P38	6	P40

LED	管脚	段码	管脚	段码	管脚
LED1	P83	LEDA	P74	LEDE	P75
LED2	P84	LEDB	P77	LEDF	P82
LED3	P85	LEDC	P78	LEDG	P73
LED4	P91	LEDD	P79	LEDP	P76

Node Name	Direction	Location	I/O Bank	VREF Group	Fitter Location
out LED1	Output	PIN_91	3	B3_N1	PIN_91
out a	Output	PIN_74	3	B3_N2	PIN_74
out b	Output	PIN_77	3	B3_N2	PIN_77
out c	Output	PIN_78	3	B3_N2	PIN_78
out d	Output	PIN_79	3	B3_N2	PIN_79
out e	Output	PIN_75	3	B3_N2	PIN_75
out f	Output	PIN_82	3	B3_N2	PIN_82
out g	Output	PIN_73	3	B3_N2	PIN_73
in w	Input	PIN_35	1	B1_N2	PIN_35
in work	Input	PIN_40	4	B4_N2	PIN_40
in x	Input	PIN_36	1	B1_N2	PIN_36
in y	Input	PIN_38	4	B4_N2	PIN_38
in z	Input	PIN_37	4	B4_N2	PIN_37

运行示例